US 20250278631A1

(19) **United States**

(12) **Patent Application Publication** (10) **Pub. No.: US 2025/0278631 A1**

VASSEUR et al. (43) **Pub. Date: Sep. 4, 2025**

(54) **MODEL-BASED ASSESSMENT OF QUALITY OF EXPERIENCE**

(71) Applicant: **Cisco Technology, Inc.**, San Jose, CA (US)

(72) Inventors: **Jean-Philippe VASSEUR**, Combloux (FR); **Grégory MERMOUD**, Venthône (CH); **Grégoire MAGENDIE**, Lamorlaye (FR); **Pierre-André SAVALLE**, Rueil-Malmaison (FR)

(21) Appl. No.: **18/593,152**

(22) Filed: **Mar. 1, 2024**

Publication Classification

(51) **Int. Cl.**

G06N 3/0895 (2023.01)

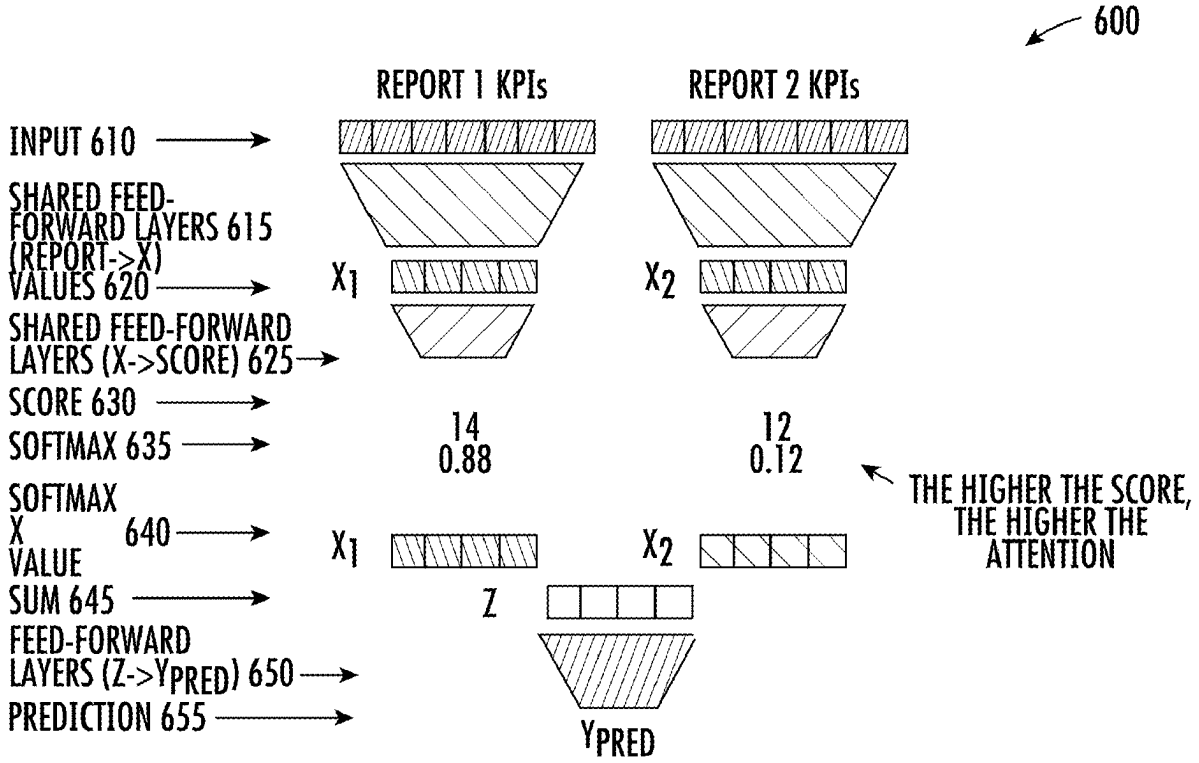
G06N 3/091 (2023.01)

(52) **U.S. Cl.**

CPC ***G06N 3/0895*** (2023.01); ***G06N 3/091*** (2023.01)

(57) **ABSTRACT**

In one embodiment, a method herein may comprise: establishing a prediction of a quality-of-experience measure from session telemetry regarding execution of an application by one or more users, the prediction established based on inputting the session telemetry into a machine learning model trained to extract attributes that drive user-based quality-of-experience feedback; determining one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model; mapping the one or more attributes of the session telemetry that significantly contributed to the prediction to a specific failure pattern from a set of known failure patterns; and mitigating the prediction of the quality-of-experience measure based on the specific failure pattern.



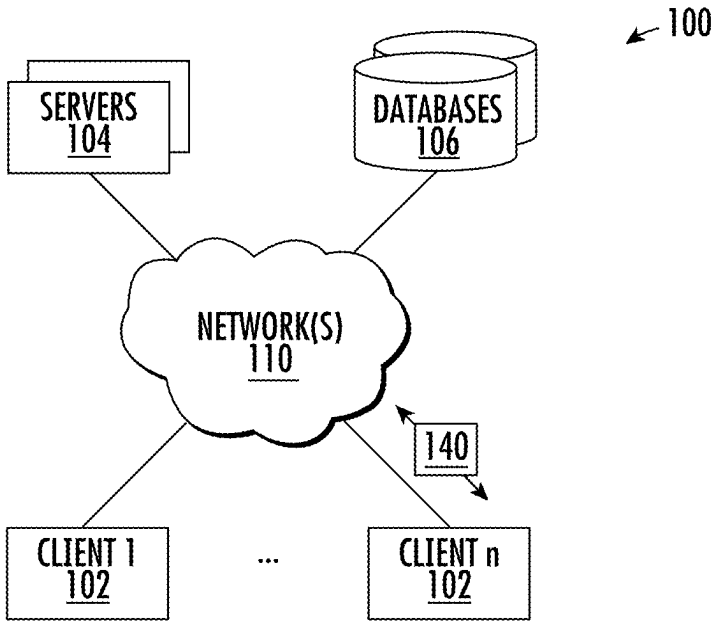

100
SERVERS
104
DATABASES
106
NETWORK(S)
110
140
CLIENT 1
102
...
CLIENT n
102

FIG. 1

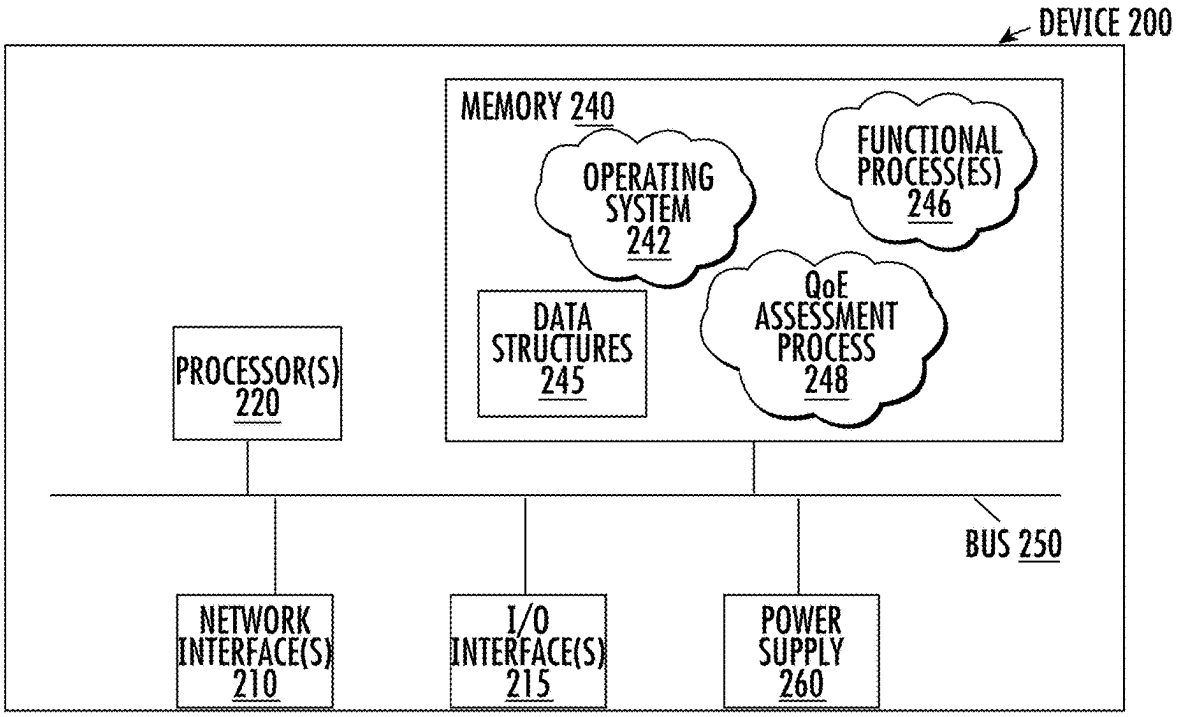

DEVICE 200
MEMORY 240
OPERATING SYSTEM 242
FUNCTIONAL PROCESS(ES) 246
DATA STRUCTURES 245
QoE ASSESSMENT PROCESS 248
PROCESSOR(S) 220
BUS 250
NETWORK INTERFACE(S) 210
I/O INTERFACE(S) 215
POWER SUPPLY 260

FIG. 2

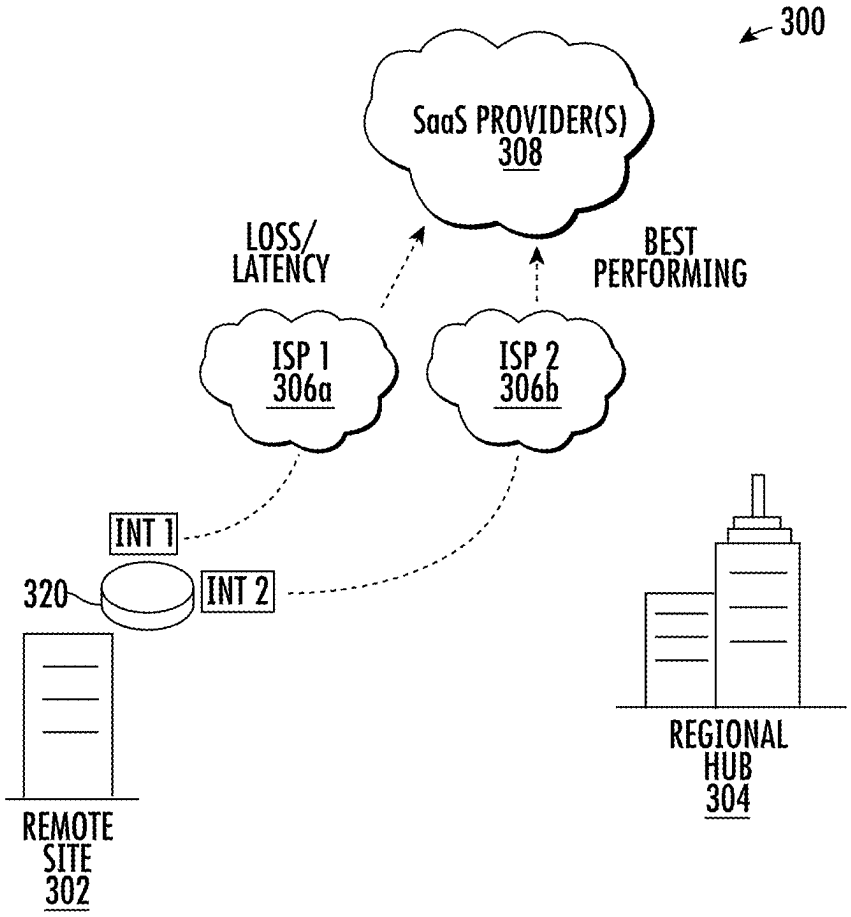

300
SaaS PROVIDER(S)
308
LOSS/
LATENCY
BEST
PERFORMING
ISP 1
306a
ISP 2
306b
INT 1
320
INT 2
REMOTE
SITE
302
REGIONAL
HUB
304

FIG. 3A

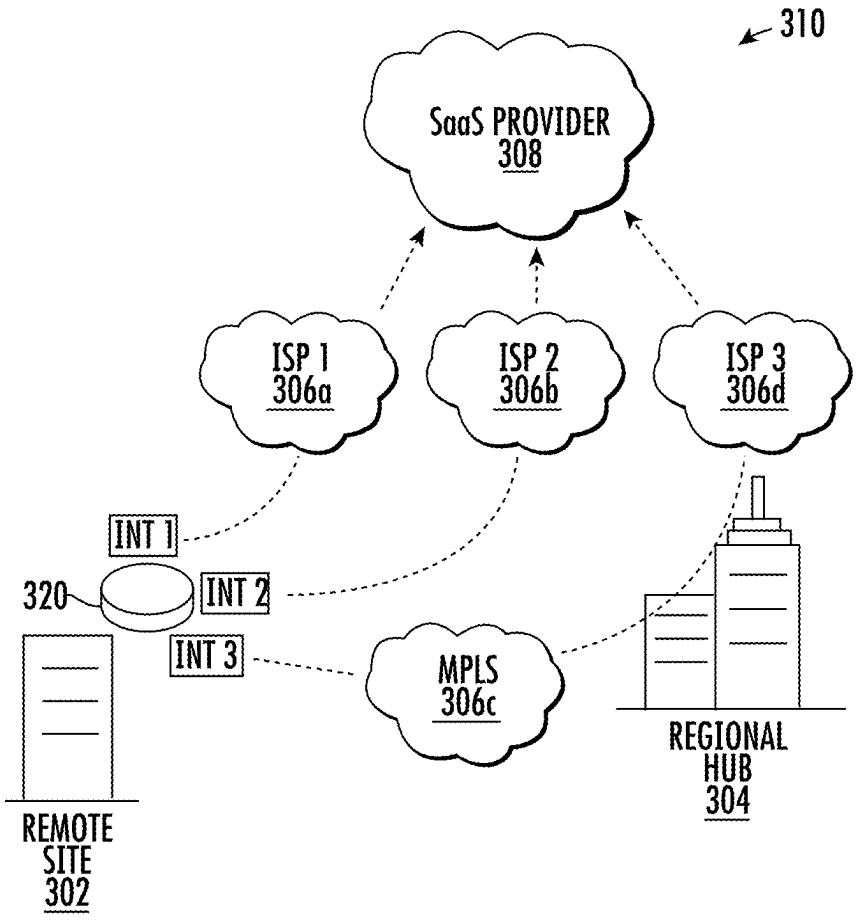

310
SaaS PROVIDER
308
ISP 1
306a
ISP 2
306b
ISP 3
306d
INT 1
INT 2
INT 3
320
MPLS
306c
REGIONAL
HUB
304
REMOTE
SITE
302

FIG. 3B

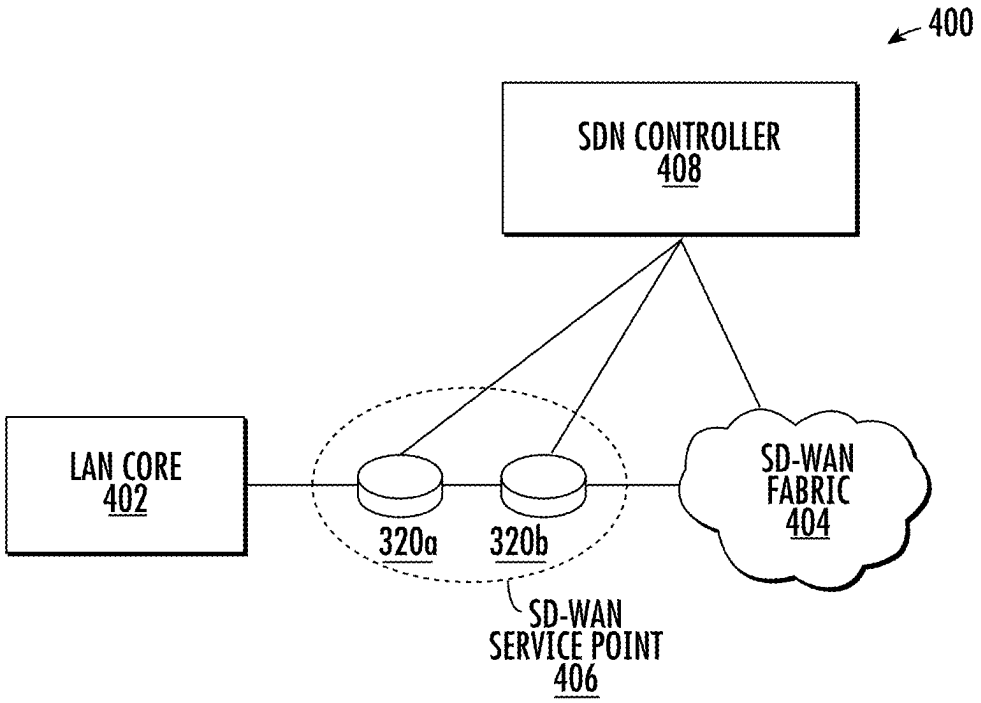

400
SDN CONTROLLER
408
LAN CORE
402
320a
320b
SD-WAN
FABRIC
404
SD-WAN
SERVICE POINT
406

FIG. 4A

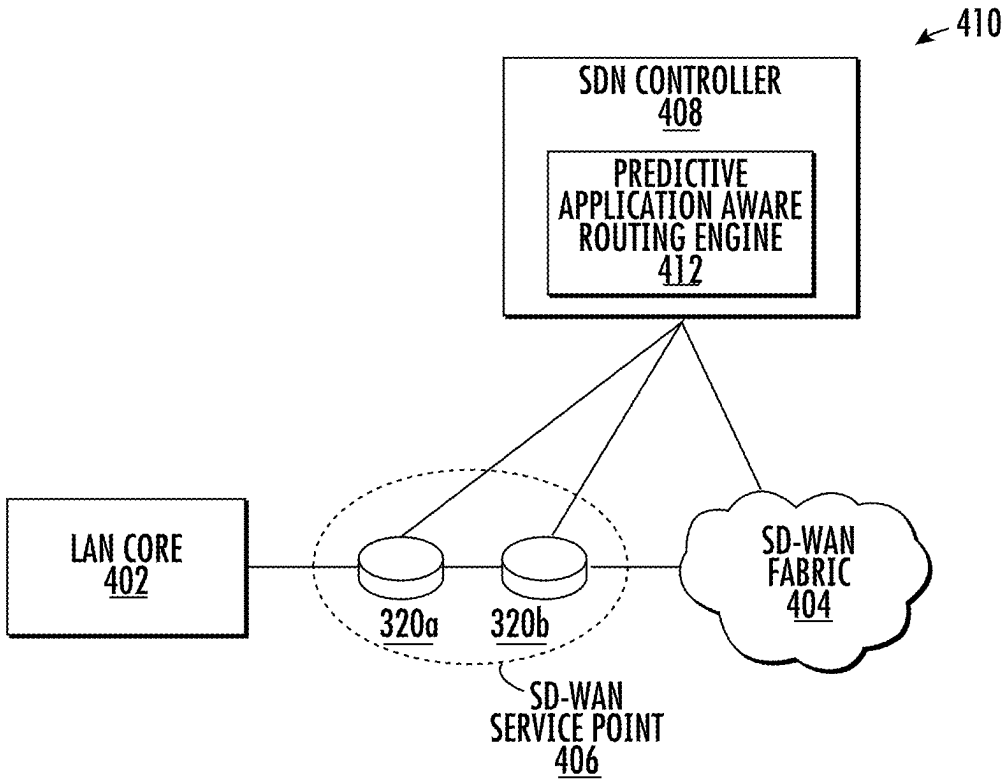

410
SDN CONTROLLER
408
PREDICTIVE
APPLICATION AWARE
ROUTING ENGINE
412
LAN CORE
402
320a
320b
SD-WAN
FABRIC
404
SD-WAN
SERVICE POINT
406

FIG. 4B

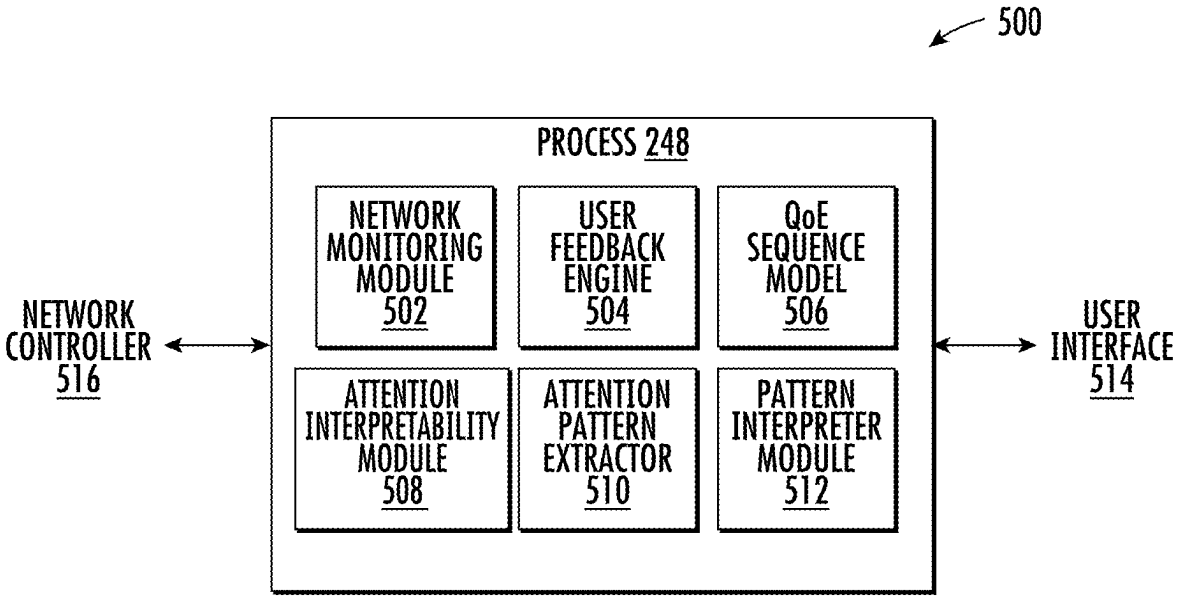

500
PROCESS 248
NETWORK MONITORING MODULE 502
USER FEEDBACK ENGINE 504
QoE SEQUENCE MODEL 506
ATTENTION INTERPRETABILITY MODULE 508
ATTENTION PATTERN EXTRACTOR 510
PATTERN INTERPRETER MODULE 512
NETWORK CONTROLLER 516
USER INTERFACE 514

FIG. 5

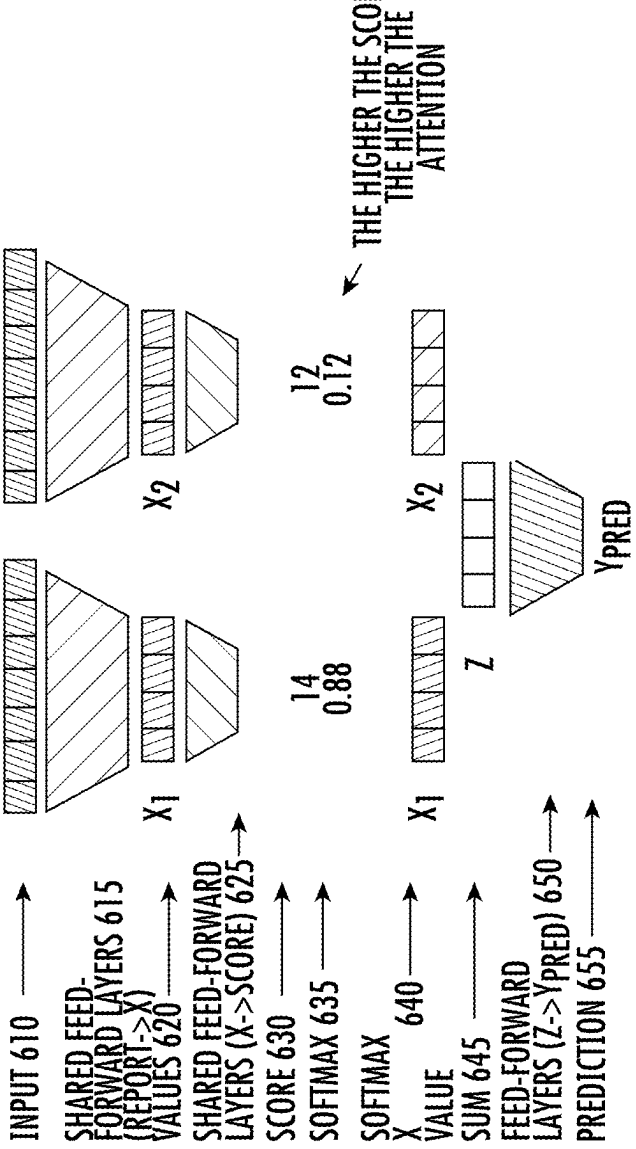

600
REPORT 1 KPIs
REPORT 2 KPIs
THE HIGHER THE SCORE, THE HIGHER THE ATTENTION
INPUT 610
SHARED FEED-FORWARD LAYERS 615 (REPORT->X)
VALUES 620
SHARED FEED-FORWARD LAYERS (X->SCORE) 625
SCORE 630
SOFTMAX 635
SOFTMAX X VALUE 640
SUM 645
FEED-FORWARD LAYERS (Z->YPRED) 650
PREDICTION 655
X1
X2
14
0.88
12
0.12
Z
YPRED

FIG. 6

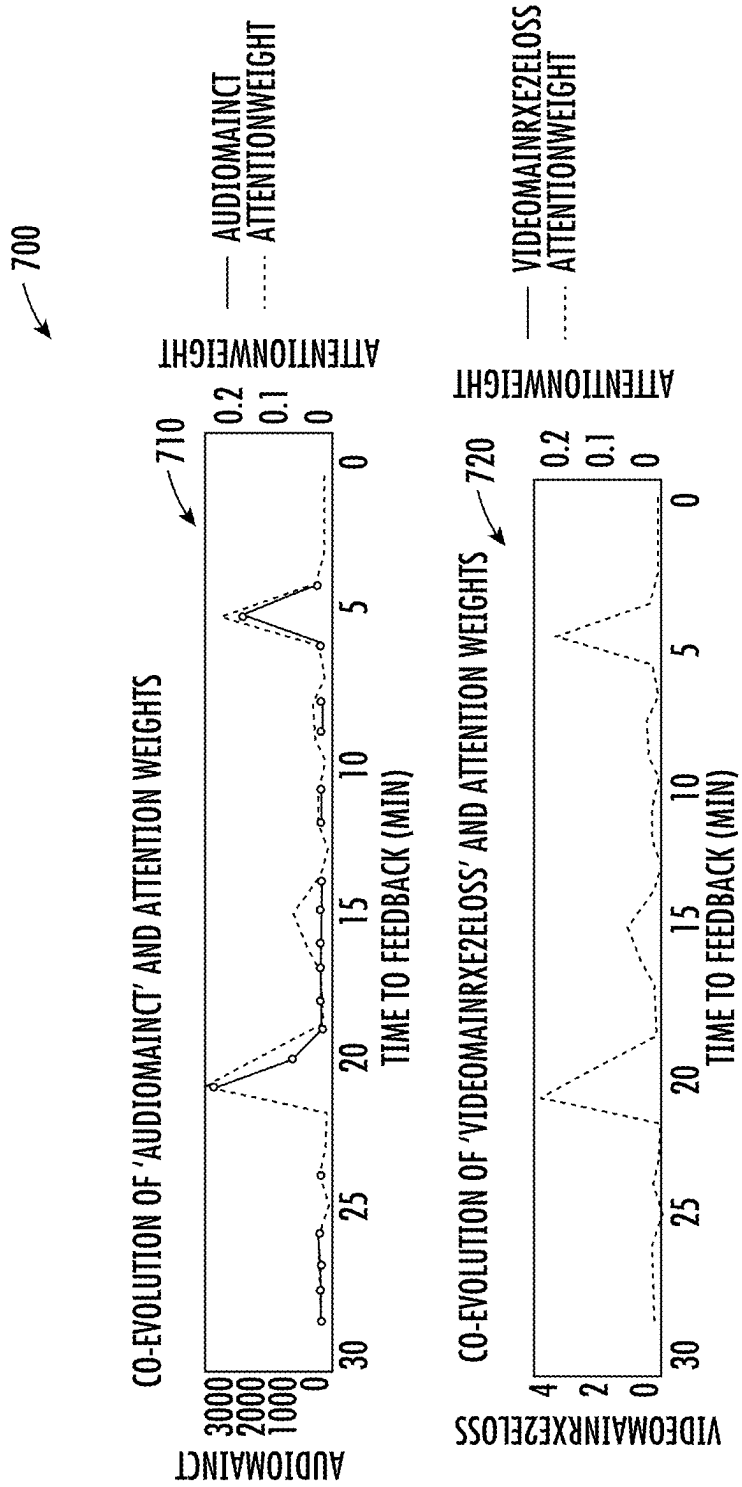

700
AUDIOMAINCT
ATTENTIONWEIGHT
710
CO-EVOLUTION OF 'AUDIOMAINCT' AND ATTENTION WEIGHTS
AUDIOMAINCT
ATTENTIONWEIGHT
TIME TO FEEDBACK (MIN)
VIDEOMAINRXE2ELOSS
ATTENTIONWEIGHT
720
CO-EVOLUTION OF 'VIDEOMAINRXE2ELOSS' AND ATTENTION WEIGHTS
VIDEOMAINRXE2ELOSS
ATTENTIONWEIGHT
TIME TO FEEDBACK (MIN)

FIG. 7A

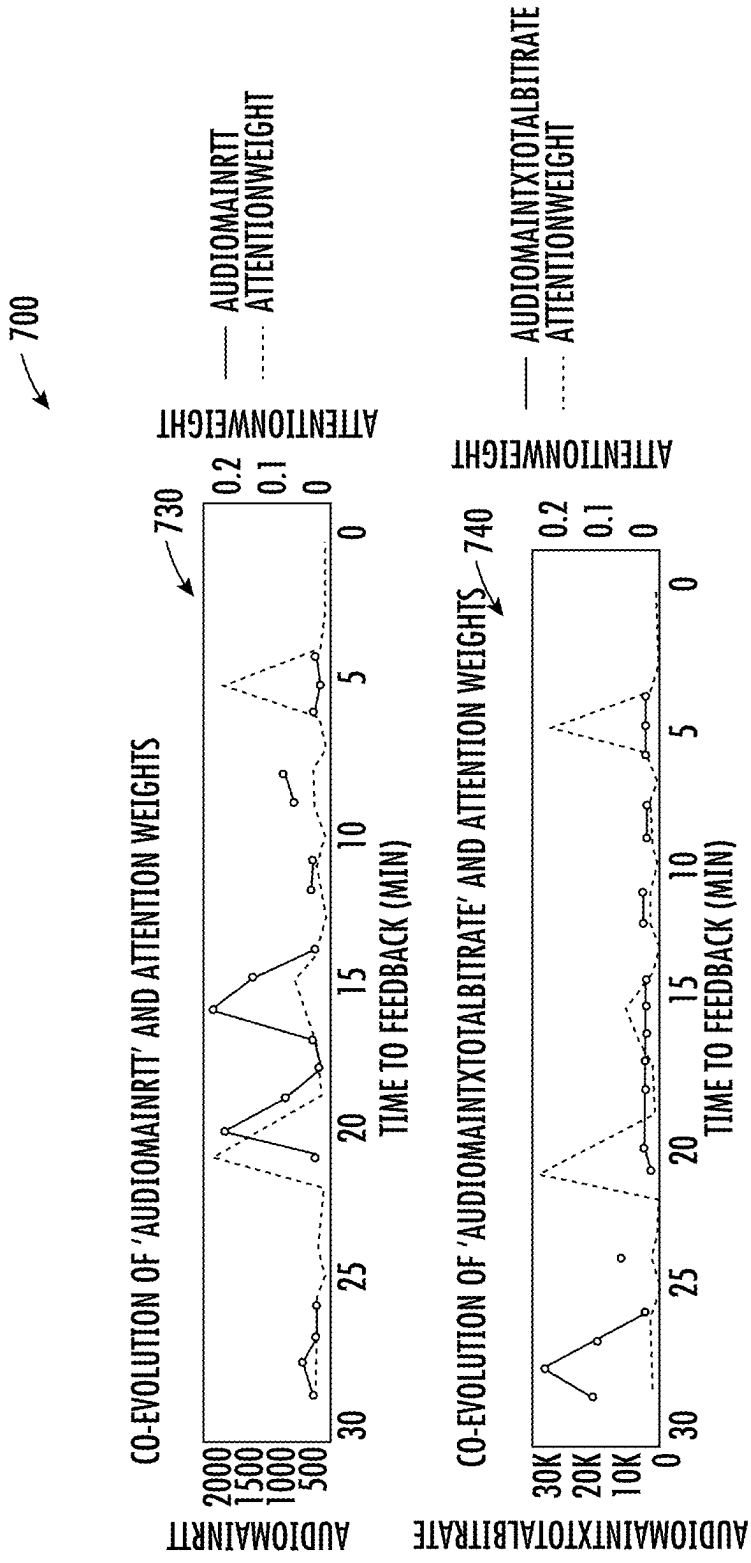

700
AUDIOMAINRTT
ATTENTIONWEIGHT
730
CO-EVOLUTION OF 'AUDIOMAINRTT' AND ATTENTION WEIGHTS
AUDIOMAINRTT
2000
1500
1000
500
ATTENTIONWEIGHT
0.2
0.1
0
0
5
10
15
20
25
30
TIME TO FEEDBACK (MIN)
AUDIOMAINTXTOTALBITRATE
ATTENTIONWEIGHT
740
CO-EVOLUTION OF 'AUDIOMAINTXTOTALBITRATE' AND ATTENTION WEIGHTS
AUDIOMAINTXTOTALBITRATE
30K
20K
10K
0
ATTENTIONWEIGHT
0.2
0.1
0
0
5
10
15
20
25
30
TIME TO FEEDBACK (MIN)

FIG. 7B

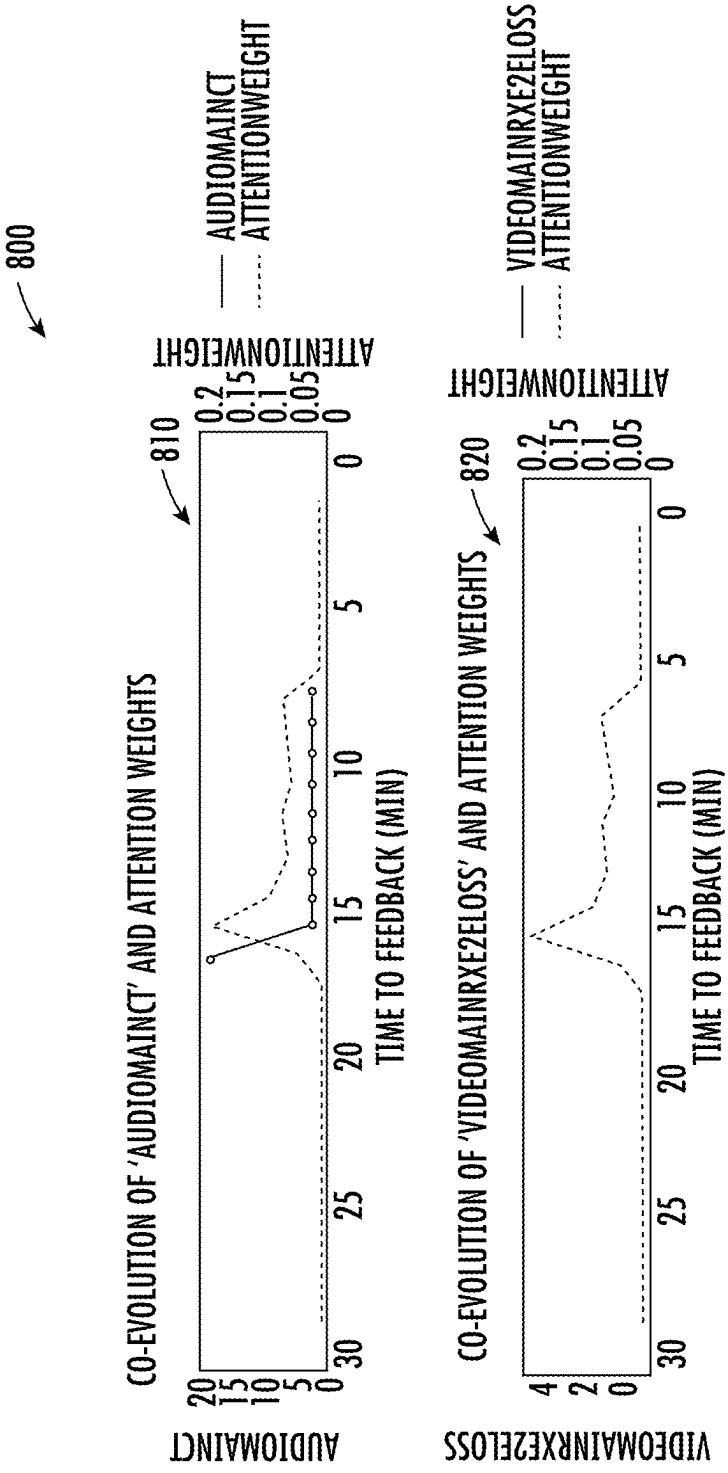

800
AUDIOMAINCT
ATTENTIONWEIGHT
VIDEOMAINRXE2ELOSS
ATTENTIONWEIGHT
ATTENTIONWEIGHT
810
820
CO-EVOLUTION OF 'AUDIOMAINCT' AND ATTENTION WEIGHTS
CO-EVOLUTION OF 'VIDEOMAINRXE2ELOSS' AND ATTENTION WEIGHTS
TIME TO FEEDBACK (MIN)
TIME TO FEEDBACK (MIN)
AUDIOMAINCT
VIDEOMAINRXE2ELOSS

FIG. 8A

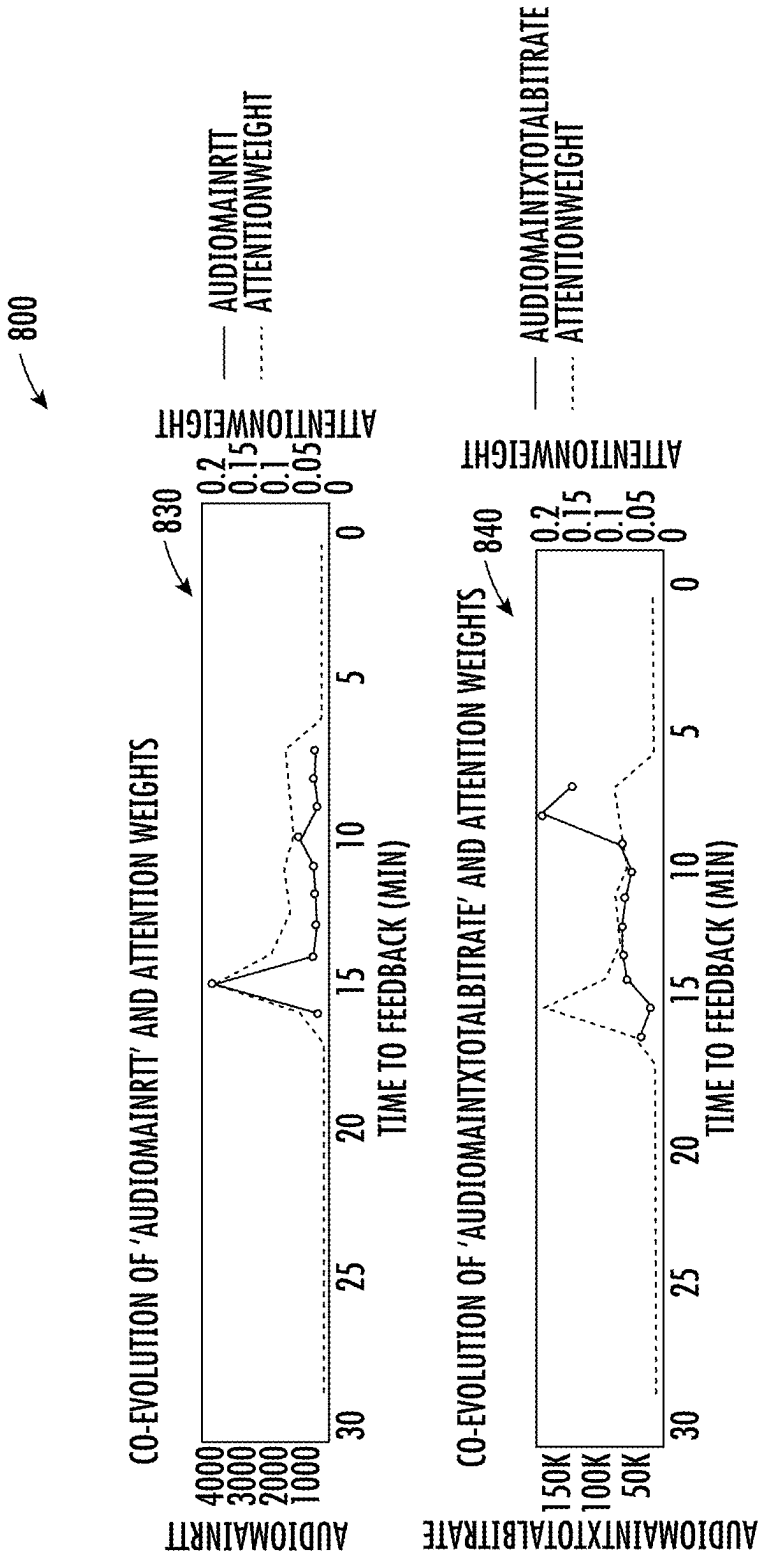

800
AUDIOMAINRTT
ATTENTIONWEIGHT
830
CO-EVOLUTION OF 'AUDIOMAINRTT' AND ATTENTION WEIGHTS
AUDIOMAINRTT
4000
3000
2000
1000
ATTENTIONWEIGHT
0.2
0.15
0.1
0.05
0
0
5
10
15
20
25
30
TIME TO FEEDBACK (MIN)
AUDIOMAINTXTOTALBITRATE
ATTENTIONWEIGHT
840
CO-EVOLUTION OF 'AUDIOMAINTXTOTALBITRATE' AND ATTENTION WEIGHTS
AUDIOMAINTXTOTALBITRATE
150K
100K
50K
ATTENTIONWEIGHT
0.2
0.15
0.1
0.05
0
0
5
10
15
20
25
30
TIME TO FEEDBACK (MIN)

FIG. 8B

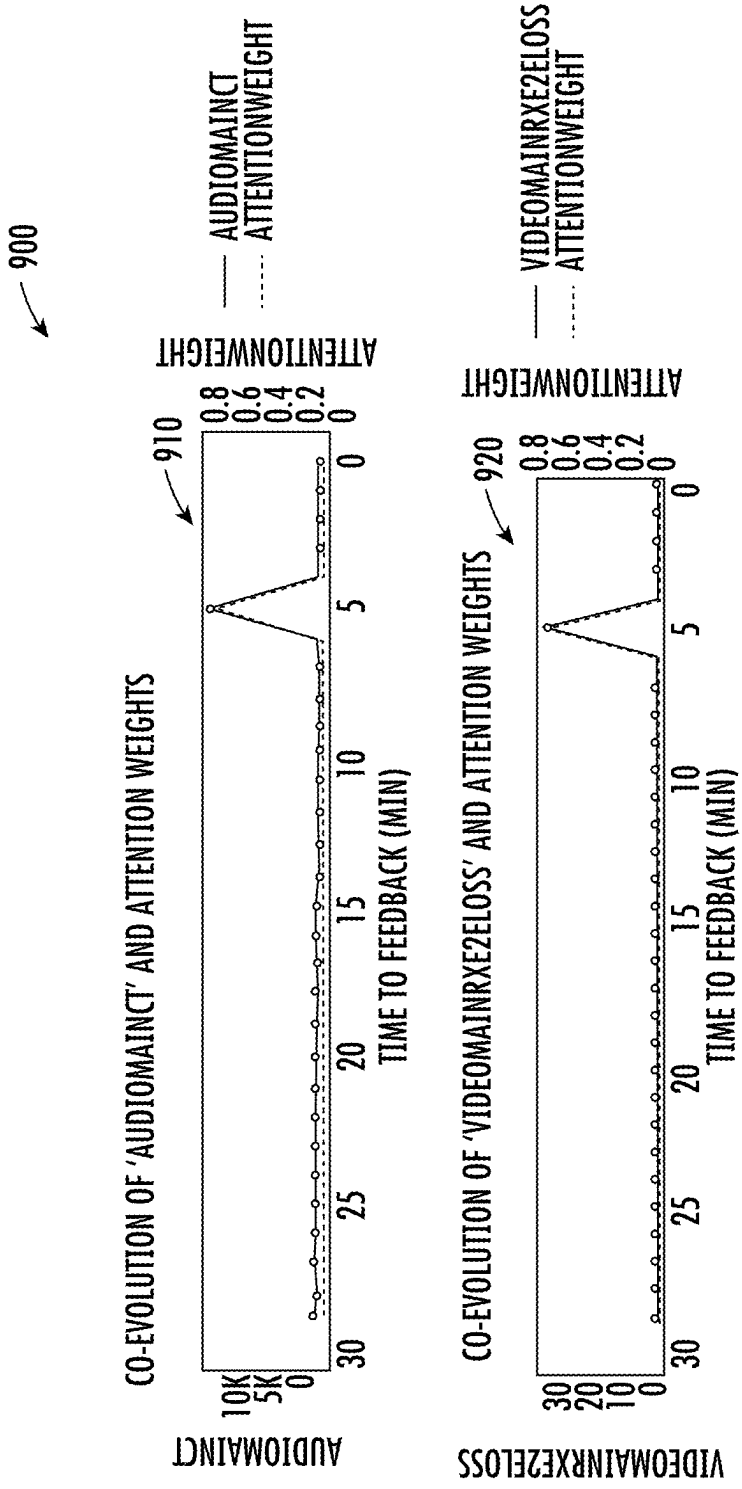

900
910
AUDIOMAINCT
ATTENTIONWEIGHT
CO-EVOLUTION OF 'AUDIOMAINCT' AND ATTENTION WEIGHTS
AUDIOMAINCT
ATTENTIONWEIGHT
TIME TO FEEDBACK (MIN)
920
VIDEOMAINRXE2ELOSS
ATTENTIONWEIGHT
CO-EVOLUTION OF 'VIDEOMAINRXE2ELOSS' AND ATTENTION WEIGHTS
VIDEOMAINRXE2ELOSS
ATTENTIONWEIGHT
TIME TO FEEDBACK (MIN)

FIG. 9A

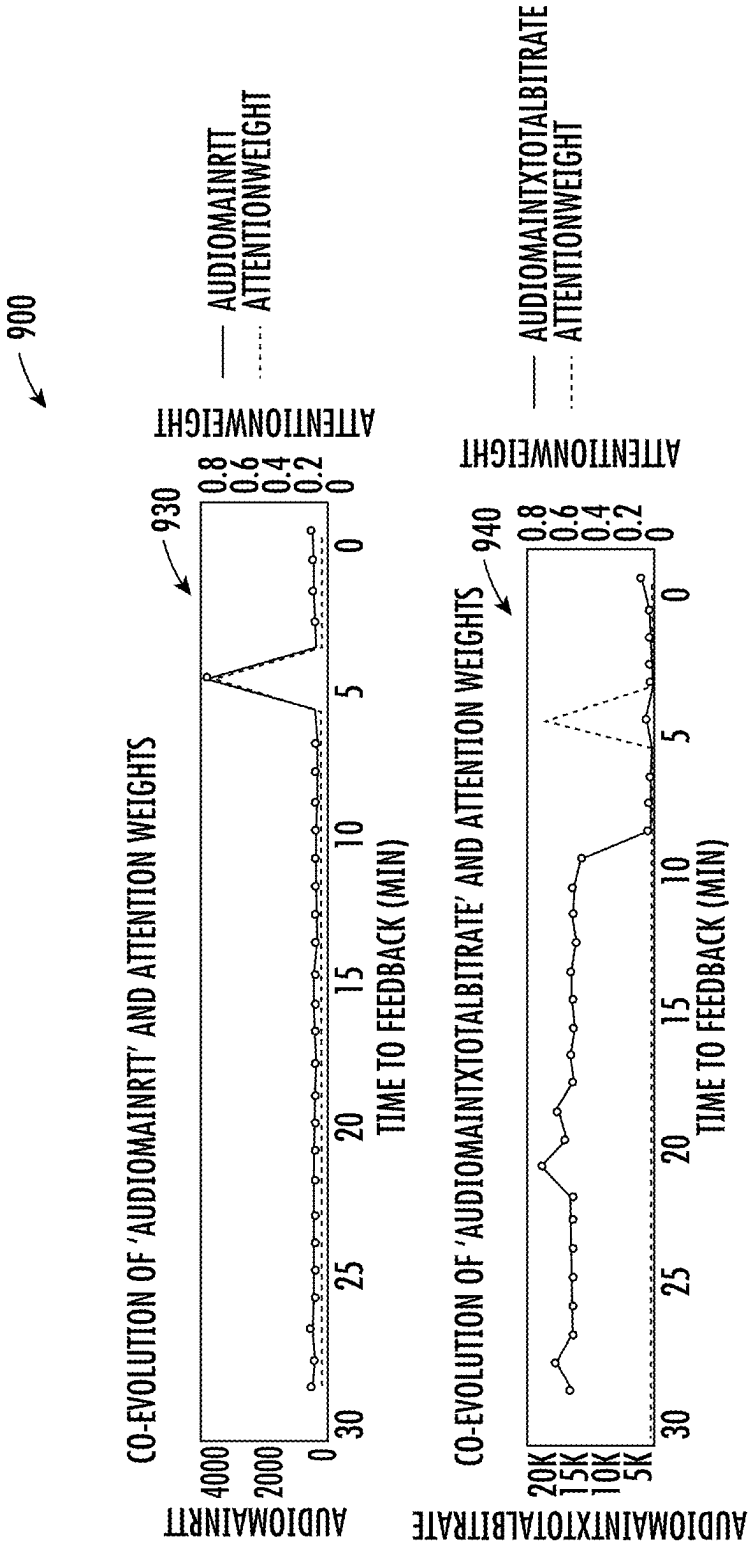

900
930
CO-EVOLUTION OF 'AUDIOMAINRTT' AND ATTENTION WEIGHTS
AUDIOMAINRTT
ATTENTIONWEIGHT
AUDIOMAINRTT
ATTENTIONWEIGHT
4000
2000
0
0.8
0.6
0.4
0.2
0
0
5
10
15
20
25
30
TIME TO FEEDBACK (MIN)
940
CO-EVOLUTION OF 'AUDIOMAINTXTOTALBITRATE' AND ATTENTION WEIGHTS
AUDIOMAINTXTOTALBITRATE
ATTENTIONWEIGHT
AUDIOMAINTXTOTALBITRATE
ATTENTIONWEIGHT
20K
15K
10K
5K
0.8
0.6
0.4
0.2
0
0
5
10
15
20
25
30
TIME TO FEEDBACK (MIN)

FIG. 9B

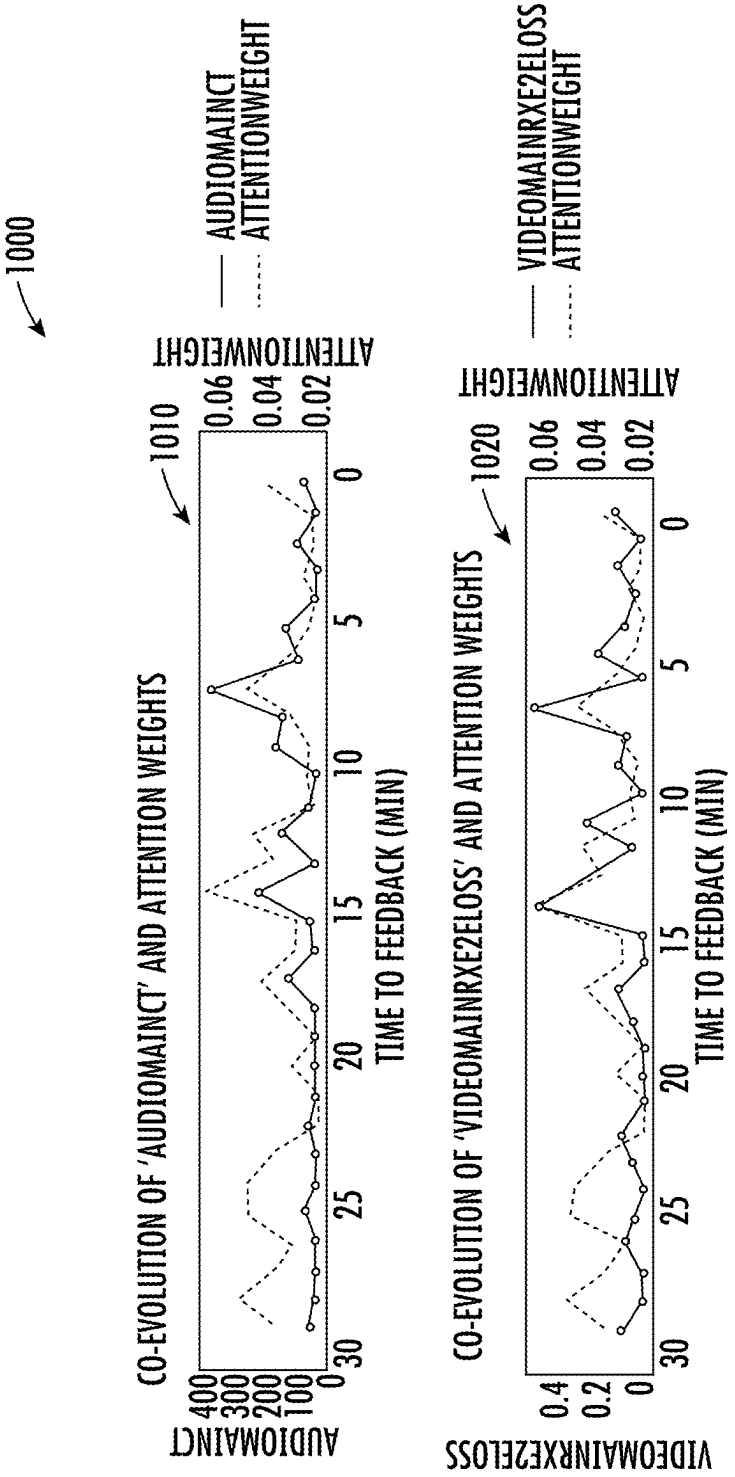

1000
AUDIOMAINCT
ATTENTIONWEIGHT
1010
CO-EVOLUTION OF 'AUDIOMAINCT' AND ATTENTION WEIGHTS
AUDIOMAINCT
ATTENTIONWEIGHT
TIME TO FEEDBACK (MIN)
VIDEOMAINRXE2ELOSS
ATTENTIONWEIGHT
1020
CO-EVOLUTION OF 'VIDEOMAINRXE2ELOSS' AND ATTENTION WEIGHTS
VIDEOMAINRXE2ELOSS
ATTENTIONWEIGHT
TIME TO FEEDBACK (MIN)

FIG. 10A

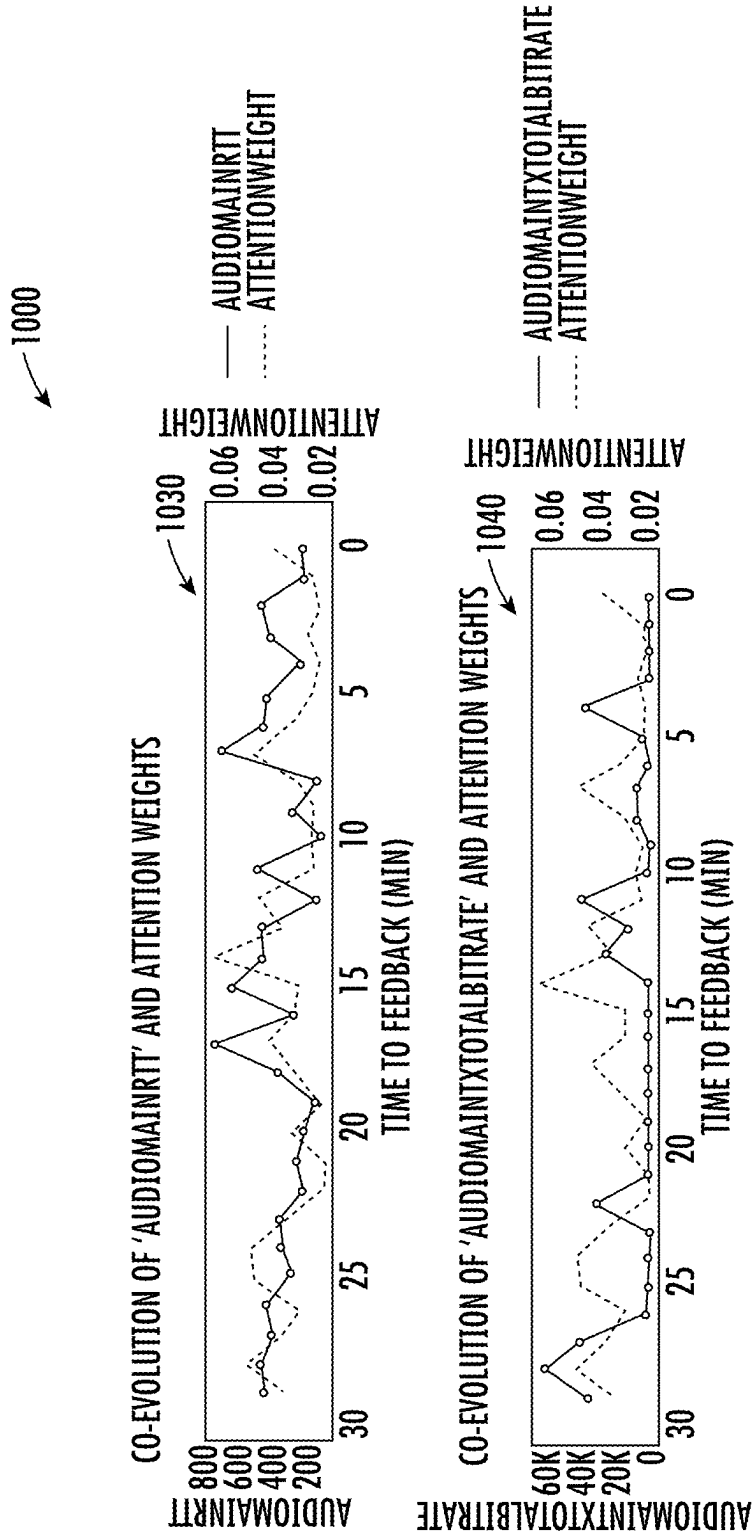

1000
AUDIOMAINRTT
ATTENTIONWEIGHT
CO-EVOLUTION OF 'AUDIOMAINRTT' AND ATTENTION WEIGHTS
1030
AUDIOMAINRTT
800
600
400
200
ATTENTIONWEIGHT
0.06
0.04
0.02
0
5
10
15
20
25
30
TIME TO FEEDBACK (MIN)
AUDIOMAINTXTOTALBITRATE
ATTENTIONWEIGHT
CO-EVOLUTION OF 'AUDIOMAINTXTOTALBITRATE' AND ATTENTION WEIGHTS
1040
AUDIOMAINTXTOTALBITRATE
60K
40K
20K
0
ATTENTIONWEIGHT
0.06
0.04
0.02
0
5
10
15
20
25
30
TIME TO FEEDBACK (MIN)

FIG. 10B

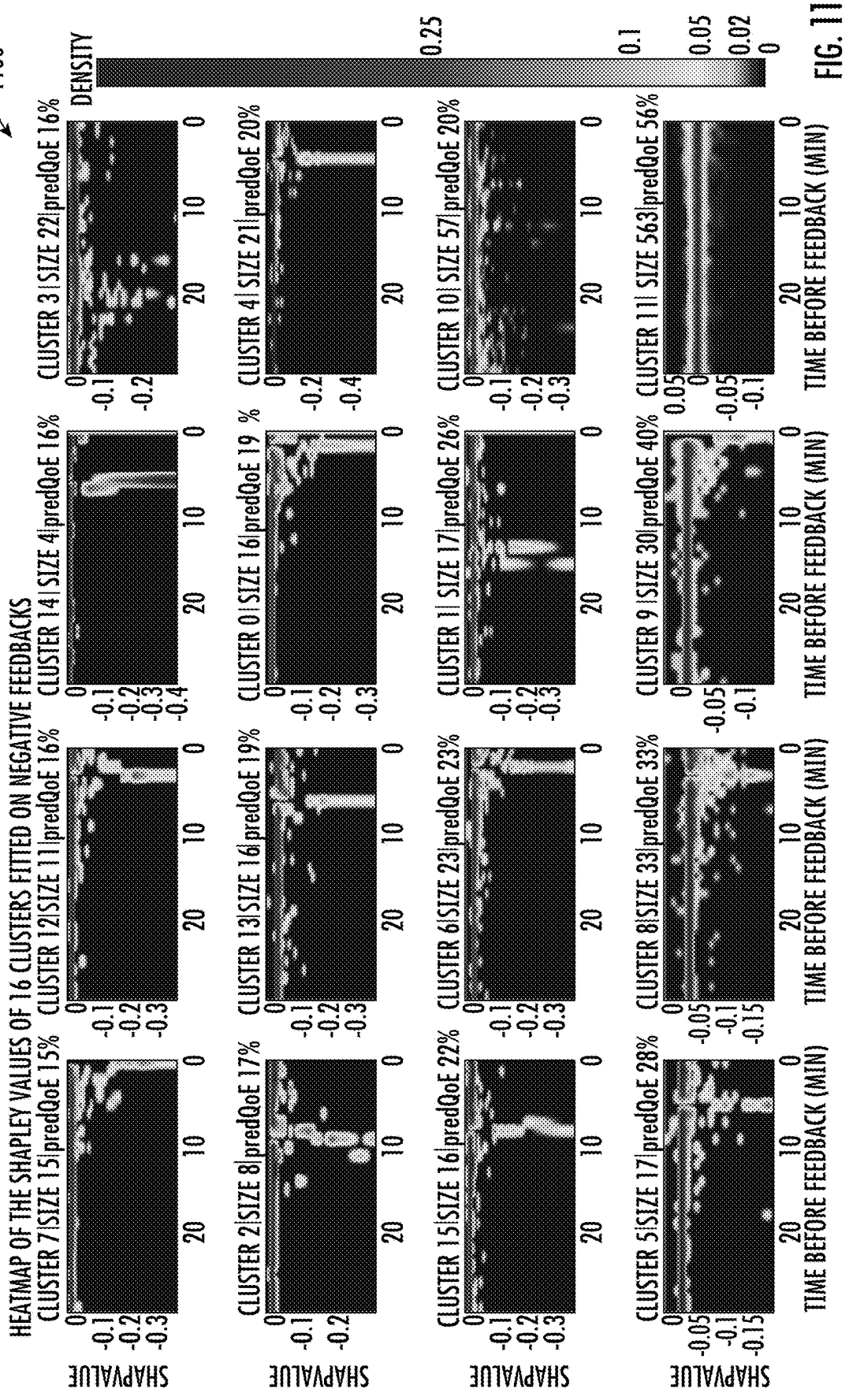

1100
HEATMAP OF THE SHAPLEY VALUES OF 16 CLUSTERS FITTED ON NEGATIVE FEEDBACKS
CLUSTER 7|SIZE 15|predQoE 15%
CLUSTER 12|SIZE 11|predQoE 16%
CLUSTER 14|SIZE 4|predQoE 16%
CLUSTER 3|SIZE 22|predQoE 16%
CLUSTER 2|SIZE 8|predQoE 17%
CLUSTER 13|SIZE 16|predQoE 19%
CLUSTER 0|SIZE 16|predQoE 19 %
CLUSTER 4|SIZE 21|predQoE 20%
CLUSTER 15|SIZE 16|predQoE 22%
CLUSTER 6|SIZE 23|predQoE 23%
CLUSTER 1|SIZE 17|predQoE 26%
CLUSTER 10|SIZE 57|predQoE 20%
CLUSTER 5|SIZE 17|predQoE 28%
CLUSTER 8|SIZE 33|predQoE 33%
CLUSTER 9|SIZE 30|predQoE 40%
CLUSTER 11|SIZE 563|predQoE 56%
SHAPVALUE
TIME BEFORE FEEDBACK (MIN)
DENSITY
0.25
0.1
0.05
0.02
0

FIG. 11

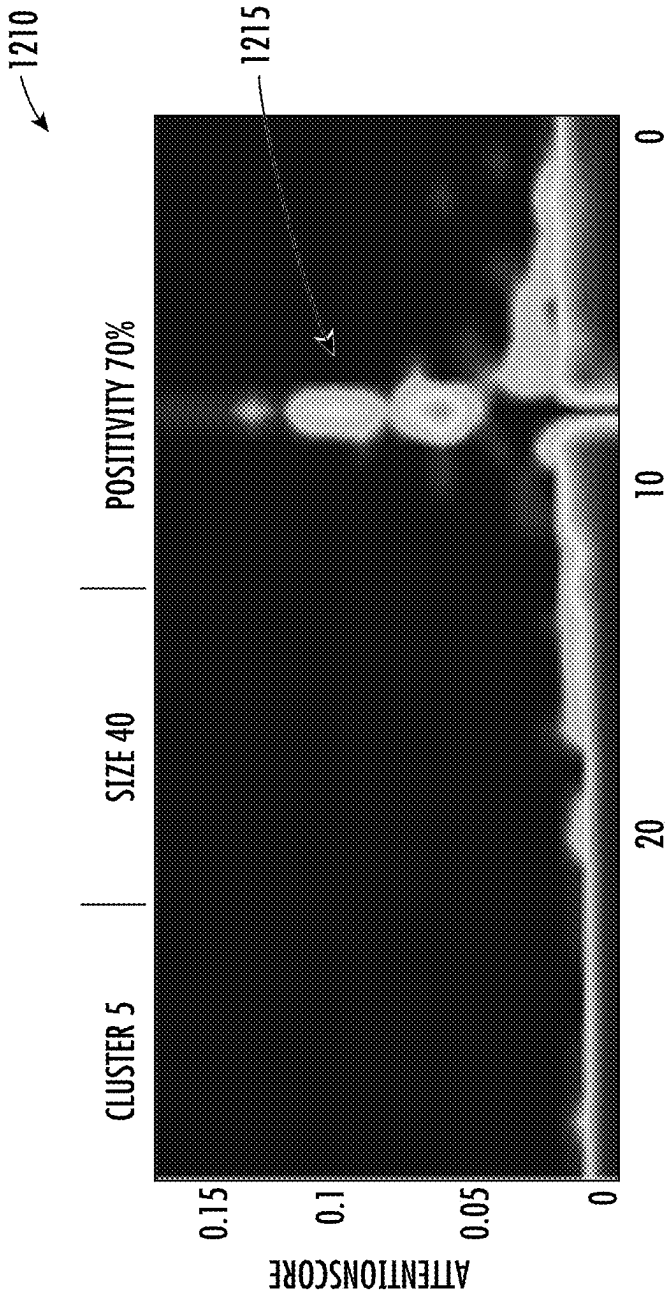

1210
1215
CLUSTER 5
SIZE 40
POSITIVITY 70%
ATTENTIONSCORE
0.15
0.1
0.05
0
0
10
20

FIG. 12A

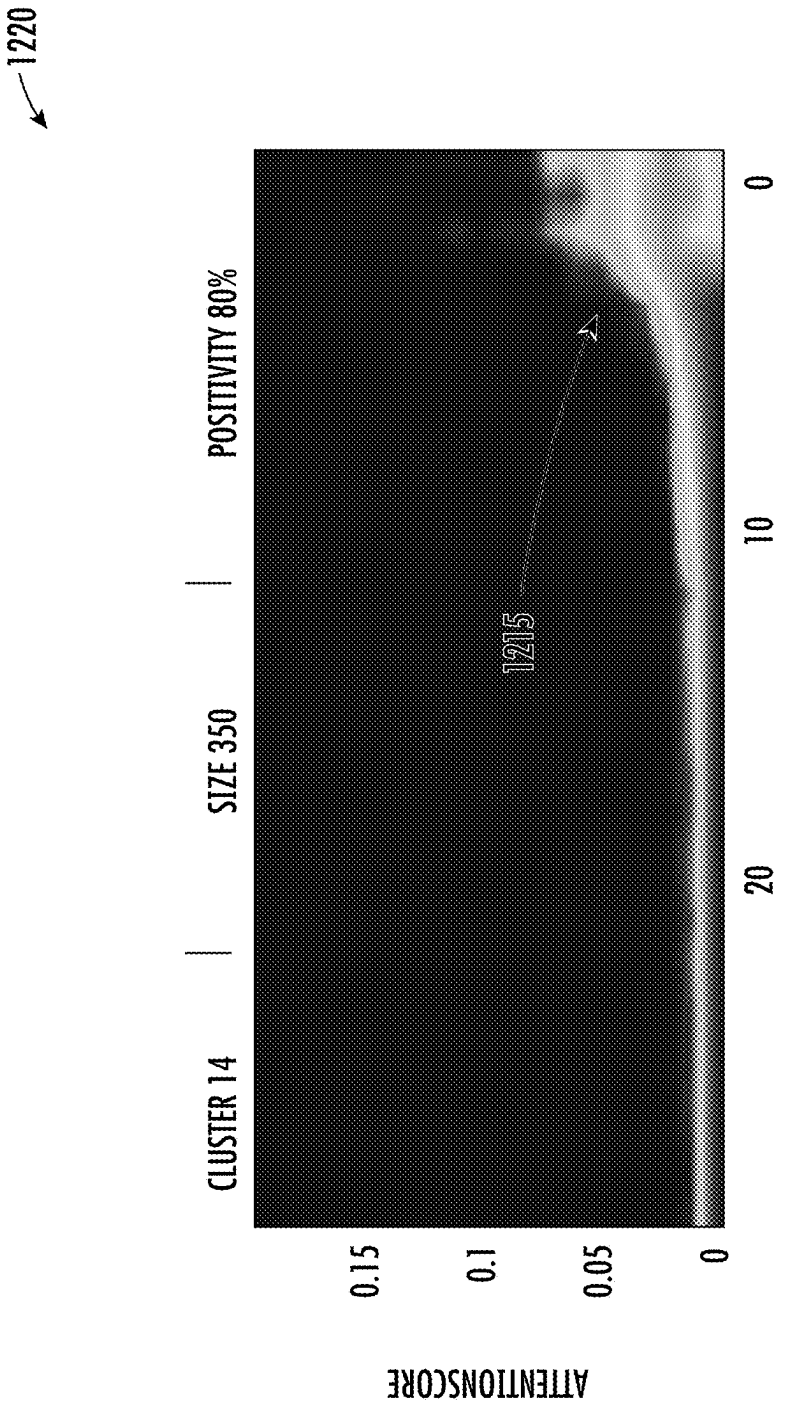

1220
CLUSTER 14
SIZE 350
POSITIVITY 80%
1215
ATTENTIONSCORE
0.15
0.1
0.05
0
0
10
20

FIG. 12B

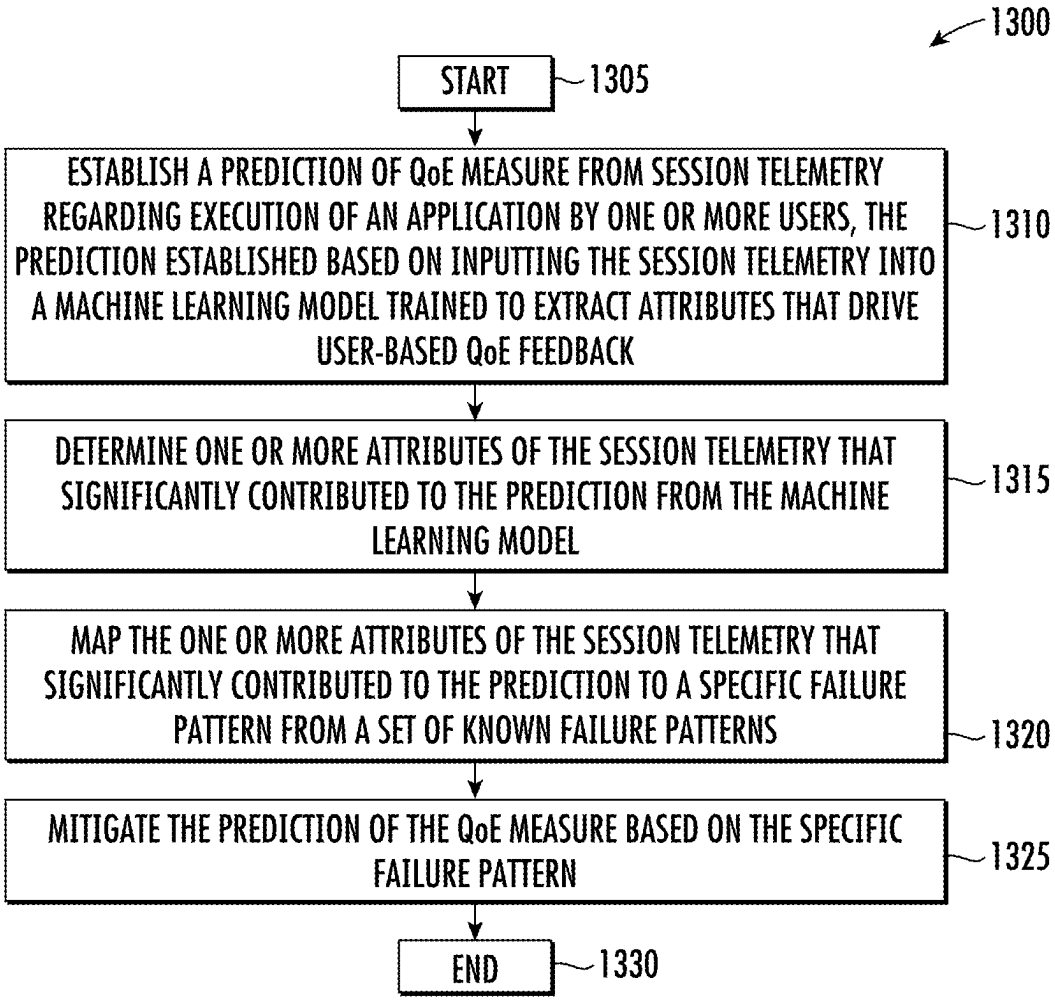

1300
START
1305
ESTABLISH A PREDICTION OF QoE MEASURE FROM SESSION TELEMETRY REGARDING EXECUTION OF AN APPLICATION BY ONE OR MORE USERS, THE PREDICTION ESTABLISHED BASED ON INPUTTING THE SESSION TELEMETRY INTO A MACHINE LEARNING MODEL TRAINED TO EXTRACT ATTRIBUTES THAT DRIVE USER-BASED QoE FEEDBACK
1310
DETERMINE ONE OR MORE ATTRIBUTES OF THE SESSION TELEMETRY THAT SIGNIFICANTLY CONTRIBUTED TO THE PREDICTION FROM THE MACHINE LEARNING MODEL
1315
MAP THE ONE OR MORE ATTRIBUTES OF THE SESSION TELEMETRY THAT SIGNIFICANTLY CONTRIBUTED TO THE PREDICTION TO A SPECIFIC FAILURE PATTERN FROM A SET OF KNOWN FAILURE PATTERNS
1320
MITIGATE THE PREDICTION OF THE QoE MEASURE BASED ON THE SPECIFIC FAILURE PATTERN
1325
END
1330

FIG. 13

MODEL-BASED ASSESSMENT OF QUALITY OF EXPERIENCE

TECHNICAL FIELD

[0001] The present disclosure relates generally to computer networks, and, more particularly, to a model-based assessment of quality of experience (QoE).

BACKGROUND

[0002] The Internet and the World Wide Web have enabled the proliferation of web services available for virtually all types of businesses or applications. Due to the accompanying complexity of the infrastructure supporting the services, it is becoming increasingly difficult to maintain the highest level of service performance and user experience to keep up with the increase in web services.

[0003] Current approaches used to correlate network and/or application failures to user experience are extremely simplistic. For the most part, they consist in computing averaged networking key performance indicator (KPI) values (or other statistical moments) and use static thresholds to conclude service level agreement (SLA) "violations". In the past, other empirical approaches have been used to assess the impact on a Mean Opinion Score (MOS), a human-judged overall quality of an experience, such as voice and video sessions. To date, however, there is no real understanding on how failure patterns in the network and/or applications actually drive user experience.

BRIEF DESCRIPTION OF THE DRA WINGS

[0004] The embodiments herein may be better understood by referring to the following description in conjunction with the accompanying drawings in which like reference numerals indicate identically or functionally similar elements, of which:

[0005] FIG. **1** illustrates an example computing system;

[0006] FIG. **2** illustrates an example network device/node;

[0007] FIGS. **3**A-**3**B illustrate example network deployments;

[0008] FIGS. **4**A-**4**B illustrate example software defined network (SDN) implementations;

[0009] FIG. **5** illustrates an example architecture for model-based assessment of quality of experience (QoE);

[0010] FIG. **6** illustrates an example of a basic attention mechanism;

[0011] FIGS. **7**A-**7**B illustrate an example comparison of attention weights to key performance indicators (KPIs) over a sequence of application reports;

[0012] FIGS. **8**A-**8**B illustrate further example comparisons of attention weights to KPIs over a sequence of application reports;

[0013] FIGS. **9**A-**9**B illustrate further example comparisons of attention weights to KPIs over a sequence of application reports;

[0014] FIGS. **10**A-**10**B illustrate further example comparisons of attention weights to KPIs over a sequence of application reports;

[0015] FIG. **11** illustrates an example heatmap of Shapely values of clusters fitted on negative feedbacks;

[0016] FIGS. **12**A-**12**B illustrate an example visualization of two failure patterns computed by an attention pattern extractor; and

[0017] FIG. **13** illustrates an example procedure for a model-based assessment of QoE.

DESCRIPTION OF EXAMPLE EMBODIMENTS

Overview

[0018] According to one or more embodiments of the disclosure, a method herein may comprise: establishing a prediction of a quality-of-experience measure from session telemetry regarding execution of an application by one or more users, the prediction established based on inputting the session telemetry into a machine learning model trained to extract attributes that drive user-based quality-of-experience feedback; determining one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model; mapping the one or more attributes of the session telemetry that significantly contributed to the prediction to a specific failure pattern from a set of known failure patterns; and mitigating the prediction of the quality-of-experience measure based on the specific failure pattern.

[0019] Other implementations are described below, and this overview is not meant to limit the scope of the present disclosure.

Description

[0020] A computer network is a geographically distributed collection of nodes interconnected by communication links and segments for transporting data between end nodes, such as personal computers and workstations, or other devices, such as sensors, etc. Many types of networks are available, ranging from local area networks (LANs) to wide area networks (WANs). LANs typically connect the nodes over dedicated private communications links located in the same general physical location, such as a building or campus. WANs, on the other hand, typically connect geographically dispersed nodes over long-distance communications links, such as common carrier telephone lines, optical lightpaths, synchronous optical networks (SONET), synchronous digital hierarchy (SDH) links, and others. The Internet is an example of a WAN that connects disparate networks throughout the world, providing global communication between nodes on various networks. Other types of networks, such as field area networks (FANs), neighborhood area networks (NANs), personal area networks (PANs), enterprise networks, etc. may also make up the components of any given computer network. In addition, a Mobile Ad-Hoc Network (MANET) is a kind of wireless ad-hoc network, which is generally considered a self-configuring network of mobile routers (and associated hosts) connected by wireless links, the union of which forms an arbitrary topology.

[0021] FIG. **1** is a schematic block diagram of an example simplified computing system (e.g., computing system **100**) illustratively comprising any number of client devices (e.g., client devices **102**, such as a first through nth client device), one or more servers (e.g., servers **104**), and one or more databases (e.g., databases **106**), where the devices may be in communication with one another via any number of networks (e.g., network(s) **110**). The one or more networks (e.g., network(s) **110**) may include, as would be appreciated, any number of specialized networking devices such as routers, switches, access points, etc., interconnected via

wired and/or wireless connections. For example, the devices shown and/or the intermediary devices in network(s) **110** may communicate wirelessly via links based on WiFi, cellular, infrared, radio, near-field communication, satellite, or the like. Other such connections may use hardwired links, e.g., Ethernet, fiber optic, etc.

[0022] The nodes/devices typically communicate over the network by exchanging discrete frames or packets of data (packets **140**) according to predefined protocols, such as the Transmission Control Protocol/Internet Protocol (TCP/IP) other suitable data structures, protocols, and/or signals. In this context, a protocol consists of a set of rules defining how the nodes interact with each other.

[0023] Network(s) **110** may include, for example, network backbones or other internetworking systems, and may include various customer edge (CE) routers interconnected with provider edge (PE) routers in order to communicate across a core network to provide connectivity between devices which may be located in different geographical areas and/or on different types of local networks (e.g., local/branch networks versus data center/cloud environments). For example, these routers may be interconnected by the public Internet, a multiprotocol label switching (MPLS) virtual private network (VPN), or the like. In some implementations, a router or a set of routers may be connected to a private network (e.g., dedicated leased lines, an optical network, etc.) or a VPN (e.g., MPLS VPN) thanks to a carrier network, via one or more links exhibiting different network and service level agreement characteristics.

[0024] Client devices **102** may include any number of user devices or end point devices configured to interface with the techniques herein. For example, client devices **102** may include, but are not limited to, desktop computers, laptop computers, tablet devices, smart phones, wearable devices (e.g., heads up devices, smart watches, etc.), set-top devices, smart televisions, Internet of Things (IoT) devices, autonomous devices, or any other form of computing device capable of participating with other devices via network(s) **110**.

[0025] Notably, in some implementations, servers **104** and/or databases **106**, including any number of other suitable devices (e.g., firewalls, gateways, and so on) may be part of a cloud-based service. In such cases, the servers and/or databases **106** may represent the cloud-based device (s) that provide certain services described herein, and may be distributed, localized (e.g., on the premise of an enterprise, or "on prem"), or any combination of suitable configurations, as will be understood in the art. Servers **104**, for example, may be configured as a network controller/supervisory service located in a data center with databases **106**, accordingly. For instance, servers **104** may include, in various implementations, a network management server (NMS), a dynamic host configuration protocol (DHCP) server, a constrained application protocol (CoAP) server, an outage management system (OMS), an application policy infrastructure controller (APIC), an application server, etc.

[0026] Those skilled in the art will also understand that any number of nodes, devices, links, etc. may be used in computing system **100**, and that the view shown herein is for simplicity. As would also be appreciated, computing system **100** may include any number of local networks, data centers, cloud environments, devices/nodes, servers, etc. Also, those skilled in the art will further understand that while the network is shown in a certain orientation, the computing system **100** is merely an example illustration that is not meant to limit the disclosure.

[0027] For instance, smart object networks, such as sensor networks, in particular, are a specific type of network (e.g., computing system **100**) having spatially distributed autonomous devices such as sensors, actuators, etc., that cooperatively monitor physical or environmental conditions at different locations, such as, e.g., energy/power consumption, resource consumption (e.g., water/gas/etc. for advanced metering infrastructure or "AMI" applications) temperature, pressure, vibration, sound, radiation, motion, pollutants, etc. Other types of smart objects include actuators, e.g., responsible for turning on/off an engine or perform any other actions. Sensor networks, a type of smart object network, are typically shared-media networks, such as wireless or PLC networks. That is, in addition to one or more sensors, each sensor device (node) in a sensor network may generally be equipped with a radio transceiver or other communication port such as PLC, a microcontroller, and an energy source, such as a battery. Generally, size and cost constraints on smart object nodes (e.g., sensors) result in corresponding constraints on resources such as energy, memory, computational speed and bandwidth.

[0028] In some implementations, the techniques herein may be applied to still other network topologies and configurations. For example, the techniques herein may be applied to peering points with high-speed links, data centers, etc.

[0029] Notably, web services can be used to provide communications between electronic and/or computing devices over a network, such as the Internet. A web site is an example of a type of web service. A web site is typically a set of related web pages that can be served from a web domain. A web site can be hosted on a web server. A publicly accessible web site can generally be accessed via a network, such as the Internet. The publicly accessible collection of web sites is generally referred to as the World Wide Web (WWW).

[0030] Also, cloud computing generally refers to the use of computing resources (e.g., hardware and software) that are delivered as a service over a network (e.g., typically, the Internet). Cloud computing includes using remote services to provide a user's data, software, and computation.

[0031] Moreover, distributed applications can generally be delivered using cloud computing techniques. For example, distributed applications can be provided using a cloud computing model, in which users are provided access to application software and databases over a network. The cloud providers generally manage the infrastructure and platforms (e.g., servers/appliances) on which the applications are executed. Various types of distributed applications can be provided as a cloud service or as a Software as a Service (SaaS) over a network, such as the Internet.

[0032] According to various implementations, a software-defined WAN (SD-WAN) may be used in computing system **100** to connect local networks and data center/cloud environments. In general, an SD-WAN uses a software defined networking (SDN)-based approach to instantiate tunnels on top of the physical network and control routing decisions, accordingly. For example, one tunnel may connect a customer edge (CE) router at the edge of a local network to router a remote CE router at the edge of a data center/cloud environment over an MPLS or Internet-based service pro-

vider network in a network backbone. Similarly, a second tunnel may also connect these routers over a 4G/5G/LTE cellular service provider network. SD-WAN techniques allow the WAN functions to be virtualized, essentially forming a virtual connection between local networks and data center/cloud environments on top of the various underlying connections. Another feature of SD-WAN is centralized management by a supervisory service that can monitor and adjust the various connections, as needed.

[0033] FIG. **2** is a schematic block diagram of an example node/device **200** (e.g., an apparatus) that may be used with one or more implementations described herein, e.g., as any of the nodes or devices shown in FIG. **1** above or described in further detail below. The device **200** may comprise one or more of the network interfaces **210** (e.g., wired, wireless, etc.), input/output interfaces (I/O interfaces **215**, inclusive of any associated peripheral devices such as displays, keyboards, cameras, microphones, speakers, etc.), at least one processor (e.g., processor(s) **220**), and a memory **240** interconnected by a system bus **250**, as well as a power supply **260** (e.g., battery, plug-in, etc.).

[0034] The network interfaces **210** include the mechanical, electrical, and signaling circuitry for communicating data over physical links coupled to the computing system **100**. The network interfaces may be configured to transmit and/or receive data using a variety of different communication protocols. Notably, a physical network interface (e.g., network interfaces **210**) may also be used to implement one or more virtual network interfaces, such as for virtual private network (VPN) access, known to those skilled in the art.

[0035] The memory **240** comprises a plurality of storage locations that are addressable by the processor(s) **220** and the network interfaces **210** for storing software programs and data structures associated with the implementations described herein. The processor(s) **220** may comprise necessary elements or logic adapted to execute the software programs and manipulate the data structures **245**. An operating system **242** (e.g., the Internetworking Operating System, or IOS®, of Cisco Systems, Inc., another operating system, etc.), portions of which are typically resident in memory **240** and executed by the processor(s), functionally organizes the node by, inter alia, invoking network operations in support of software processors and/or services executing on the device. These software processors and/or services may comprise one or more functional processes **246**, and on certain devices, a Quality of Experience (QoE) assessment process (process **248**), as described herein, each of which may alternatively be located within individual network interfaces.

[0036] Notably, one or more functional processes **246**, when executed by processor(s) **220**, cause each device **200** to perform the various functions corresponding to the particular device's purpose and general configuration. For example, a router would be configured to operate as a router, a server would be configured to operate as a server, an access point (or gateway) would be configured to operate as an access point (or gateway), a client device would be configured to operate as a client device, and so on.

[0037] For instance, one or more functional processes **246** may include computer executable instructions executed by the processor(s) **220** to perform routing functions in conjunction with one or more routing protocols. These functions may, on capable devices, be configured to manage a routing/forwarding table (a data structure **245**) containing, e.g., data used to make routing/forwarding decisions. In various cases, connectivity may be discovered and known, prior to computing routes to any destination in the network, e.g., link state routing such as Open Shortest Path First (OSPF), or Intermediate-System-to-Intermediate-System (ISIS), or Optimized Link State Routing (OLSR). For instance, paths may be computed using a shortest path first (SPF) or constrained shortest path first (CSPF) approach. Conversely, neighbors may first be discovered (e.g., a priori knowledge of network topology is not known) and, in response to a needed route to a destination, send a route request into the network to determine which neighboring node may be used to reach the desired destination. Example protocols that take this approach include Ad-hoc On-demand Distance Vector (AODV), Dynamic Source Routing (DSR), DYnamic MANET On-demand Routing (DYMO), etc. Notably, on devices not capable or configured to store routing entries, the one or more functional processes **246** may consist solely of providing mechanisms necessary for source routing techniques. That is, for source routing, other devices in the network can tell the less capable devices exactly where to send the packets, and the less capable devices simply forward the packets as directed.

[0038] In various implementations, as detailed further below, one or more functional processes **246** and/or QoE assessment process (process **248**) may include computer executable instructions that, when executed by processor(s) **220**, cause device **200** to perform the techniques described herein. To do so, in some implementations, one or more functional processes **246** and/or process **248** may utilize machine learning. In general, machine learning is concerned with the design and the development of techniques that take as input empirical data (such as network statistics and performance indicators) and recognize complex patterns in these data. One very common pattern among machine learning techniques is the use of an underlying model M, whose parameters are optimized for minimizing the cost function associated to M, given the input data. For instance, in the context of classification, the model M may be a straight line that separates the data into two classes (e.g., labels) such that M=a*x+b*y+c and the cost function would be the number of misclassified points. The learning process then operates by adjusting the parameters a, b, c such that the number of misclassified points is minimal. After this optimization phase (or learning phase), model M can be used very easily to classify new data points. Often, M is a statistical model, and the cost function is inversely proportional to the likelihood of M, given the input data.

[0039] In various implementations, one or more functional processes **246** and/or process **248** may employ one or more supervised, unsupervised, or semi-supervised machine learning models. Generally, supervised learning entails the use of a training set of data, as noted above, that is used to train the model to apply labels to the input data. For example, the training data may include sample network observations that do, or do not, violate a given network health status rule and are labeled as such. On the other end of the spectrum are unsupervised techniques that do not require a training set of labels. Notably, while a supervised learning model may look for previously seen patterns that have been labeled as such, an unsupervised model may instead look to whether there are sudden changes in the

behavior. Semi-supervised learning models take a middle ground approach that uses a greatly reduced set of labeled training data.

[0040] Example machine learning techniques that one or more functional processes **246** and/or process **248** can employ may include, but are not limited to, nearest neighbor (NN) techniques (e.g., k-NN models, replicator NN models, etc.), statistical techniques (e.g., Bayesian networks, etc.), clustering techniques (e.g., k-means, mean-shift, etc.), neural networks (e.g., reservoir networks, artificial neural networks, etc.), support vector machines (SVMs), generative adversarial networks (GANs), long short-term memory (LSTM), logistic or other regression, Markov models or chains, principal component analysis (PCA) (e.g., for linear models), singular value decomposition (SVD), multi-layer perceptron (MLP) artificial neural networks (ANNs) (e.g., for non-linear models), replicating reservoir networks (e.g., for non-linear models, typically for timeseries), random forest classification, or the like.

[0041] In further implementations, one or more functional processes **246** and/or process **248** may also include one or more generative artificial intelligence/machine learning models. In contrast to discriminative models that simply seek to perform pattern matching for purposes such as anomaly detection, classification, or the like, generative approaches instead seek to generate new content or other data (e.g., audio, video/images, text, etc.), based on an existing body of training data. For instance, in the context of network assurance, one or more functional processes **246** and/or process **248** may use a generative model to generate synthetic network traffic based on existing user traffic to test how the network reacts. Example generative approaches can include, but are not limited to, generative adversarial networks (GANs), large language models (LLMs), other transformer models, and the like. In some instances, one or more functional processes **246** and/or process **248** may be executed to intelligently route LLM workloads across executing nodes (e.g., communicatively connected GPUs clustered into domains).

[0042] The performance of a machine learning model can be evaluated in a number of ways based on the number of true positives, false positives, true negatives, and/or false negatives of the model. For example, the false positives of the model may refer to the number of times the model incorrectly predicted whether a network health status rule was violated. Conversely, the false negatives of the model may refer to the number of times the model predicted that a health status rule was not violated when, in fact, the rule was violated. True negatives and positives may refer to the number of times the model correctly predicted whether a rule was violated or not violated, respectively. Related to these measurements are the concepts of recall and precision. Generally, recall refers to the ratio of true positives to the sum of true positives and false negatives, which quantifies the sensitivity of the model. Similarly, precision refers to the ratio of true positives to the sum of true and false positives.

[0043] It will be apparent to those skilled in the art that other processor and memory types, including various computer-readable media, may be used to store and execute program instructions pertaining to the techniques described herein. Also, while the description illustrates various processes, it is expressly contemplated that various processes may be implemented as modules configured to operate in accordance with the techniques herein (e.g., according to the functionality of a similar process). Further, while processes may be shown and/or described separately, those skilled in the art will appreciate that processes may be routines or modules within other processes.

[0044] As noted above, in software defined WANs (SD-WANs), traffic between individual sites are sent over tunnels. The tunnels are configured to use different switching fabrics, such as MPLS, Internet, 4G or 5G, etc. Often, the different switching fabrics provide different quality of service (QOS) at varied costs. For example, an MPLS fabric typically provides high QoS when compared to the Internet, but is also more expensive than traditional Internet. Some applications requiring high QoS (e.g., video conferencing, voice calls, etc.) are traditionally sent over the more costly fabrics (e.g., MPLS), while applications not needing strong guarantees are sent over cheaper fabrics, such as the Internet.

[0045] Traditionally, network policies map individual applications to Service Level Agreements (SLAs), which define the satisfactory performance metric(s) for an application, such as loss, latency, or jitter. Similarly, a tunnel is also mapped to the type of SLA that is satisfies, based on the switching fabric that it uses. During runtime, the SD-WAN edge router then maps the application traffic to an appropriate tunnel. Currently, the mapping of SLAs between applications and tunnels is performed manually by an expert, based on their experiences and/or reports on the prior performances of the applications and tunnels.

[0046] The emergence of infrastructure as a service (IaaS) and software-as-a-service (SaaS) is having a dramatic impact of the overall Internet due to the extreme virtualization of services and shift of traffic load in many large enterprises. Consequently, a branch office or a campus can trigger massive loads on the network.

[0047] FIGS. **3A-3B** illustrate example network deployments (network deployment **300**, network deployment **310**, respectively). As shown, a router **320** located at the edge of a remote site **302** may provide connectivity between a local area network (LAN) of the remote site **302** and one or more cloud-based, SaaS provider(s) **308**. For example, in the case of an SD-WAN, router **320** may provide connectivity to SaaS provider(s) **308** via tunnels across any number of networks **306**. This allows clients located in the LAN of remote site **302** to access cloud applications (e.g., Office 365™, Dropbox™, etc.) served by SaaS provider(s) **308**.

[0048] As would be appreciated, SD-WANs allow for the use of a variety of different pathways between an edge device and a SaaS provider. For example, as shown in network deployment **300** in FIG. **3A**, router **320** may utilize two Direct Internet Access (DIA) connections to connect with SaaS provider(s) **308**. More specifically, a first interface of router **320** (e.g., a network interface **210**, described previously), Int **1**, may establish a first communication path (e.g., a tunnel) with SaaS provider(s) **308** via a first Internet Service Provider (ISP) **306***a*, denoted ISP **1** in FIG. **3A**. Likewise, a second interface of router **320**, Int **2**, may establish a backhaul path with SaaS provider(s) **308** via a second ISP **306***b*, denoted ISP **2** in FIG. **3A**.

[0049] FIG. **3B** illustrates another network deployment **310** in which Int **1** of router **320** at the edge of remote site **302** establishes a first path to SaaS provider(s) **308** via ISP **1** and Int **2** establishes a second path to SaaS provider(s) **308** via a second ISP **306***b*. In contrast to the example in FIG. **3A**, Int **3** of router **320** may establish a third path to SaaS

provider(s) **308** via a private corporate network **306***c* (e.g., an MPLS network) to a private data center or regional hub **304** which, in turn, provides connectivity to SaaS provider (s) **308** via another network, such as a third ISP **306***d*.

[0050] Regardless of the specific connectivity configuration for the network, a variety of access technologies may be used (e.g., ADSL, 4G, 5G, etc.) in all cases, as well as various networking technologies (e.g., public Internet, MPLS (with or without strict SLA), etc.) to connect the LAN of remote site **302** to SaaS provider(s) **308**, Other deployments scenarios are also possible, such as using Colo, accessing SaaS provider(s) **308** via Zscaler or Umbrella services, and the like.

[0051] FIG. **4**A illustrates an example SDN implementation **400**, according to various embodiments. As shown, there may be a LAN core **402** at a particular location, such as remote site **302** shown previously in FIGS. **3**A-**3**B. Connected to LAN core **402** may be one or more routers that form an SD-WAN service point **406** which provides connectivity between LAN core **402** and SD-WAN fabric **404**. For instance, SD-WAN service point **406** may comprise routers **110***a*-**110***b*.

[0052] Overseeing the operations of routers **110***a*-**110***b* in SD-WAN service point **406** and SD-WAN fabric **404** may be an SDN controller **408**. In general, SDN controller **408** may comprise one or more devices (e.g., a device **200**) configured to provide a supervisory service (e.g., one or more functional processes **246**), typically hosted in the cloud, to SD-WAN service point **406** and SD-WAN fabric **404**. For instance, SDN controller **408** may be responsible for monitoring the operations thereof, promulgating policies (e.g., security policies, etc.), installing or adjusting IPsec routes/tunnels between LAN core **402** and remote destinations such as regional hub **304** and/or SaaS provider(s) **308** in FIGS. **3**A-**3**B, and the like.

[0053] As noted above, a primary networking goal may be to design and optimize the network to satisfy the requirements of the applications that it supports. So far, though, the two worlds of "applications" and "networking" have been fairly siloed. More specifically, the network is usually designed in order to provide the best SLA in terms of performance and reliability, often supporting a variety of Class of Service (CoS), but unfortunately without a deep understanding of the actual application requirements. On the application side, the networking requirements are often poorly understood even for very common applications such as voice and video for which a variety of metrics have been developed over the past two decades, with the hope of accurately representing the Quality of Experience (QoE) from the standpoint of the users of the application.

[0054] More and more applications are moving to the cloud and many do so by leveraging a SaaS model. Consequently, the number of applications that became network-centric has grown approximately exponentially with the raise of SaaS applications, such as Office **365**, ServiceNow, SAP, voice, and video, to mention a few. All of these applications rely heavily on private networks and the Internet, bringing their own level of dynamicity with adaptive and fast changing workloads. On the network side, SD-WAN provides a high degree of flexibility allowing for efficient configuration management using SDN controllers with the ability to benefit from a plethora of transport access (e.g., MPLS, Internet with supporting multiple CoS, LTE, satellite links, etc.), multiple classes of service and policies to reach private and public networks via multi-cloud SaaS.

[0055] Furthermore, the level of dynamicity observed in today's network has never been so high. Millions of paths across thousands of Service Provides (SPs) and a number of SaaS applications have shown that the overall QoS(s) of the network in terms of delay, packet loss, jitter, etc. drastically vary with the region, SP, access type, as well as over time with high granularity. The immediate consequence is that the environment is highly dynamic due to:

[0056] New in-house applications being deployed;

[0057] New SaaS applications being deployed everywhere in the network, hosted by a number of different cloud providers;

[0058] Internet, MPLS, LTE transports providing highly varying performance characteristics, across time and regions;

[0059] SaaS applications themselves being highly dynamic: it is common to see new servers deployed in the network. DNS resolution allows the network for being informed of a new server deployed in the network leading to a new destination and a potentially shift of traffic towards a new destination without being even noticed.

[0060] According to various implementations, SDN controller **408** may employ application aware routing, which refers to the ability to route traffic so as to satisfy the requirements of the application, as opposed to exclusively relying on the (constrained) shortest path to reach a destination IP address.

[0061] In particular, various attempts have been made to extend the notion of routing, CSPF, link state routing protocols (ISIS, OSPF, etc.) using various metrics (e.g., Multi-topology Routing) where each metric would reflect a different path attribute (e.g., delay, loss, latency, etc.), but each time with a static metric. At best, current approaches rely on SLA templates specifying the application requirements so as for a given path (e.g., a tunnel) to be "eligible" to carry traffic for the application. In turn, application SLAs are checked using regular probing. Other solutions compute a metric reflecting a particular network characteristic (e.g., delay, throughput, etc.) and then selecting the supposed 'best path,' according to the metric.

[0062] The term 'SLA failure' refers to a situation in which the SLA for a given application, often expressed as a function of delay, loss, or jitter, is not satisfied by the current network path for the traffic of a given application. This leads to poor QoE from the standpoint of the users of the application. Modern SaaS solutions like Viptela, CloudonRamp SaaS, and the like, allow for the computation of per application QoE by sending HyperText Transfer Protocol (HTTP) probes along various paths from a branch office and then route the application's traffic along a path having the best QoE for the application. At a first sight, such an approach may solve many problems. Unfortunately, though, there are several shortcomings to this approach:

[0063] The SLA for the application is 'guessed,' using static thresholds.

[0064] Routing is still entirely reactive: decisions are made using probes that reflect the status of a path at a given time, in contrast with the notion of an informed decision.

[0065] SLA failures are very common in the Internet and a good proportion of them could be avoided (e.g., using an alternate path), if predicted in advance,

[0066] In various embodiments, the techniques herein allow for a predictive application aware routing engine to be deployed, such as in the cloud, to control routing decisions in a network. For instance, the predictive application aware routing engine may be implemented as part of an SDN controller (e.g., SDN controller **408**) or other supervisory service, or may operate in conjunction therewith. For instance, FIG. **4**B illustrates an example **410** in which SDN controller **408** includes a predictive application aware routing engine **412** (e.g., through execution of QoE assessment process, process **248**). Further embodiments provide for predictive application aware routing engine **412** to be hosted on a router or at any other location in the network.

[0067] During execution, predictive application aware routing engine **412** makes use of a high volume of network and application telemetry (e.g., from routers **320***a*-**320***b*, SD-WAN fabric **404**, etc.) so as to compute statistical and/or machine learning models to control the network with the objective of optimizing the application experience and reducing potential down times. To that end, predictive application aware routing engine **412** may compute a variety of models to understand application requirements, and predictably route traffic over private networks and/or the Internet, thus optimizing the application experience while drastically reducing SLA failures and downtimes.

[0068] In other words, predictive application aware routing engine **412** may first predict SLA violations in the network that could affect the QoE of an application (e.g., due to spikes of packet loss or delay, sudden decreases in bandwidth, etc.). In other words, predictive application aware routing engine **412** may use SLA violations as a proxy for actual QoE information (e.g., ratings by users of an online application regarding their perception of the application), unless such QoE information is available from the provider of the online application. In turn, predictive application aware routing engine **412** may then implement a corrective measure, such as rerouting the traffic of the application, prior to the predicted SLA violation. For instance, in the case of video applications, it now becomes possible to maximize throughput at any given time, which is of utmost importance to maximize the QoE of the video application. Optimized throughput can then be used as a service triggering the routing decision for specific application requiring highest throughput, in one embodiment. In general, routing configuration changes are also referred to herein as routing "patches," which are typically temporary in nature (e.g., active for a specified period of time) and may also be application-specific (e.g., for traffic of one or more specified applications).

[0069] As noted above, predictive networking engines, such as predictive application aware routing engine **412**, seek to select the best path from among a plurality of paths P**1**, P**2**, . . . , PN such that end users of a given online application, either SaaS-delivered (e.g., WebEx, Zoom, O365, Salesforce, SAP, etc.) or datacenter-hosted (and monitored via tools such as Datadog, AppDynamics, etc.) have the best experience possible. In the context of SD-WAN, these paths may be probed for liveness and basic path QoS metrics (e.g., loss, latency, jitter, throughput, etc.) at the network level (L3), typically using technologies such as Bidirectional Forwarding Detection (BFD) probing.

[0070] However, actively probing the QoS metrics of the network paths reveals little about the actual experience of the end user. Indeed, while the path performance may be considered degraded from a networking perspective, an end user of an application may not even notice a change in their overall application experience (e.g., due to the codecs in use by the application, the ability of the application to adapt to network problems, etc.). As a result, networks today are primarily optimized using metrics such as mean opinion score (MOS) metrics, that are only vague approximations or proxies of what is thought to be the real end user experience. Furthermore, such proxies do not account at all for the inherently subjective nature of the application experience, which may be perceived differently by different users, and are not customized in any way to the individual end users. Said differently, there is a very poor understanding today of what the actual experience of an application user is.

[0071] In particular, in recent years, enterprise networks have been undergoing a fundamental transformation where users and applications have become increasingly distributed while technologies (such as SD-WAN) have enabled unprecedented flexibility in terms of network architecture and underlay connectivity options.

[0072] At the same time, collaboration applications ~critical for day-to-day business operations, have moved from on-premises deployment to a SaaS Cloud delivery model which allows application vendors and rapidly deploy and take advantage of the latest and most novel techniques and codecs that can be used to increase robustness of media content.

[0073] In this highly dynamic environment, the ability of network administrators to understand the impact of network performance (or lack of) on media applications quality of experience (QoE) and ensuring Service Level Agreements (SLAs) is becoming increasingly challenging.

[0074] How much do developers know about "user experience" and "networking actions that should be taken in order to improve user experience"? For decades the answer to the first question has been entirely relying on network Key Performance Indicator (KPIs) such as delay, loss, and jitter for which hard boundaries should not be exceeded in order to meet the application SLA. In the example of voice, the usual SLA boundaries are 150 ms for Delay, 50 ms for Jitter and a maximum of 3% of packet loss. Unfortunately, such values are highly debatable. Moreover, the measurements granularity is usually left unspecified making the values totally irrelevant. A path experiencing a constant delay of 120 ms for voice over a period of 10 minutes provides a very different user experience than a path with the same average delay that keeps varying between 20 ms and 450 ms . . . The dynamics of such KPIs is even more critical for packet loss and jitter in the case of voice and video traffic (e.g., 10 s of 80% packet loss would severely impact the user experience although averaged out over 10 s would give a low value totally acceptable according to the threshold). Without a doubt, the user experience requires a more subtle and accurate approach to determine the networking requirements a path should meet in order to maximize the user satisfaction, capturing local phenomenon (e.g., effects on delay, jitter, and loss at higher frequencies) but also telemetry from upper layers (applications). For years the concept of layers isolation has been a core principle of the Internet. Such an approach allowed for avoiding layer dependency (e.g., often referred to as layer violation) at a time where several

protocols and technologies were developed at each layer, thus enabling the design and deployment of new layers (e.g. PHY, MAC, etc.) independent of each other, allowing the Internet to scale. Still, with modern applications requiring tight SLAs a cross-layer approach is highly desirable. The answer to the second question is equally challenging and remains unanswered. Although the effect of specific actions at a given layer of the networking stack on user experience can be qualitatively evaluated, being able to precisely quantify it is often unknown: determining that voice quality is low along a highly congested path may be relatively easy but by how much should the bandwidth be increased or the weight of the queue used for voice be tuned in order to increase the user experience score?

[0075] Cognitive Networks introduce a new approach, where instead of taking a siloed approach where networking systems poorly understand user satisfaction, focus on a single layer, and poorly connect with networking actions, Cognitive Networks are fully driven by understanding user experience (cognition) using cross-layer telemetry and ground truth user feedback in order to determine which networking actions can optimize the user experience. To that end a rich set of telemetry sources are gathered along with labeled user feedback to train Machine Learning model used to predict (for forecast) the user experience (aka QoE). Such a holistic approach end-to-end across layers is a paradigm shift to how networks have been designed and operated since the early days of the Internet.

Model-Based Assessment of QoE As noted above, current approaches used to correlate network and/or application failures to user experience are extremely simplistic. For the most part, they consist in computing averaged networking key performance indicator (KPI) values (or other statistical moments) and use static thresholds to conclude service level agreement (SLA) "violations". In the past, other empirical approaches have been used to assess the impact on a Mean Opinion Score (MOS), a human-judged overall quality of an experience, such as voice and video sessions. To date, however, there is no real understanding on how failure patterns in the network and/or applications actually drive user experience.

[0076] The techniques herein, on the other hand, provide a model-based assessment of quality of experience (QoE). In particular, as described in greater detail below the techniques herein allow for understanding failure patterns governing user experience. QoE models are used (e.g., GBT and Deep Neural Networks with attention mechanisms) and interpretation using Shapley values and attention weight are used to extract the key attributes that drive QoE experience. Such failure patterns are then correlated with network KPI (events) so as to adjust the network design (e.g., protection/restoration, QoE, routing policies, etc.) and improve the overall user experience.

[0077] Specifically, according to one or more embodiments of the disclosure as described in detail below, a method herein may comprise: establishing a prediction of a quality-of-experience measure from session telemetry regarding execution of an application by one or more users, the prediction established based on inputting the session telemetry into a machine learning model trained to extract attributes that drive user-based quality-of-experience feedback; determining one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model; mapping the one or more attributes of the session telemetry that significantly contributed to the prediction to a specific failure pattern from a set of known failure patterns; and mitigating the prediction of the quality-of-experience measure based on the specific failure pattern.

[0078] FIG. **5** illustrates an example architecture (architecture **500**) for model-based assessment of QoE, according to various implementations. At the core of architecture **500** is QoE assessment process (process **248**), which may be executed by a controller for a network or another device in communication therewith. For instance, process **248** may be executed by a controller for a network (e.g., SDN controller **408** in FIGS. **4A-4B**, a network controller in a different type of network, etc.), a particular networking device in the network (e.g., a router, a firewall, etc.), another device or service in communication therewith, or the like. In some embodiments, for instance, QoE assessment process (process **248**) may be used to implement a predictive application aware routing engine, such as predictive application aware routing engine **412**, or another supervisory service for the network. In other embodiments, QoE assessment process (process **248**) may be used to implement a reactive routing approach in the network, e.g., in conjunction with one or more functional processes **246**, as described above.

[0079] As shown, QoE assessment process (process **248**) may include any or all of the following components: network monitoring module **502**, a user feedback engine **504**, a QoE sequence model **506**, or "QSM", an attention interpretability module **508**, or "AIM", an attention pattern extractor **510**, or "APE", and/or a pattern interpreter module **512**, or "PIM". As would be appreciated, the functionalities of these components may be combined or omitted, as desired (e.g., implemented as part of process **248**). In addition, these components may be implemented on a singular device or in a distributed manner, in which case the combination of executing devices can be viewed as their own singular device for purposes of executing a process **248**. Also, QoE assessment process (process **248**) may be in communication with one or more user interfaces **514** and one or more network controllers **516**, accordingly.

[0080] Operationally, the techniques herein may rely on an external user feedback dataset or mechanism to collect user feedback on application quality-of-experience. This dataset combines network and application telemetry with real user feedback, also referred to as "labels". The data can be collected by polling the users about their subjective experience during an application session and then matching each collected label with the session telemetry recorded prior to the feedback. For example, certain implementations of the techniques herein may collect direct user experience metrics for an online application via a chatbot, which may be integrated directly into the online application (e.g., Webex, Slack, WhatsApp bots, etc.). In other words, a chatbot may act as a relay between the network and end users of a given application, querying live feedback about their experience.

[0081] Illustratively, certain aspects of a system in accordance with the techniques herein may associate one or more performance metrics with a particular session of an online application, and may obtain feedback from users regarding their application experience, such as by causing a chatbot to query the user for feedback regarding their application experience. The techniques herein may then associate the feedback from the user regarding their application experience with the one or more performance metrics.

[0082] In particular, network monitoring module **502** may monitor the traffic of users of an online application in (near) real-time (e.g., continuously), in some embodiments. In general, the role of network monitoring module **502** is to identify when users of a given online application are active and, optionally, obtain a (rough) assessment of the quality of service provided by the network for their sessions. To this end, network monitoring module **502** may typically be hosted on a network device (a router, wireless LAN controller, etc.) or a network controller (e.g., DNA Center, vManage, etc.) and process network telemetry related to the application traffic and path QoS. For instance, such telemetry may take the form of NetFlow records, BFD probing results, IP-SLA information, or the like. In further embodiments, another form of telemetry that network monitoring module **502** may obtain could be application-level metrics measured by the application itself. For instance, such application-level metrics may include metrics such as application-measured loss, latency, jitter, concealment time, codec statistics, audio/video bitrates, or the like. For instance, the application itself may provide this telemetry to network monitoring module **502** via an application programming interface (API) or other mechanism. In some instances, network monitoring module **502** may also be cloud-hosted and process the application-level telemetry, directly. Further embodiments provide for multiple network monitoring modules to be implemented at different locations, as well.

[0083] User feedback engine **504**, in various embodiments, may be responsible for collecting user feedback in a variety of manners, such as by presenting a chatbot to the user interface of select users, or by a simple rating, ranking, or "thumbs up" or "thumbs down" vote. In various embodiments, such a feedback engine may be integrated directly into the online application itself or presented via a separate agent or other mechanism. For instance, assume that the online application is Webex. In such a case, user feedback engine **504** may cause a chatbot or other feedback mechanism to be displayed to a selected videoconference user.

[0084] User feedback engine **504** may use different strategies to collect the feedback, such as any or all of the following, in various embodiments:

- **[0085]** Classical surveying with a 1-to-5 stars rating or similar. This is useful in case of random surveys and/or when the system has a poor estimate of the experience (e.g., upon bootstrapping the system).
- **[0086]** Direct yes/no question—e.g., "It seems you are having a very bad time with O365. Am I correct?" This is useful when the system has a good estimate of the user experience and wants to establish trust. Ideally, such queries should be accompanied of a tip or piece of advice to resolve the situation such as "You may improve your experience by switching to cellular," "You are now using a 5G link, are you seeing any improvement in term of experience during this call?" or the like.

[0087] In further embodiments, user feedback engine **504** may also leverage natural language interactions, either initiated by the chatbot or not.

[0088] Once user feedback engine **504** has collected the actual user feedback regarding their application experiences, it may associate this information with the telemetry obtained by network monitoring module **502**. For instance, in the case of process **248** being used to implement predictive application aware routing engine **412**, shown previously, the feedback could be used as ground truth information for purposes of training a predictive model to predict whether the application experience is acceptable or not, given the network-level and/or application-level telemetry that is available. Such associations can also be used for purposes of presenting information to a network operator (e.g., by showing the operator the effects of a configuration change or event on the application experience of users, etc.).

[0089] Optionally, network monitoring module **502** may also include state collector agent, which collects and stores more detailed state information about networking devices (e.g., edge devices, routers, switches) at the time a feedback request was sent by user feedback engine **504** to a user, in some embodiments. Such states are often of the utmost importance for models in charge of predicting and/or forecasting application QoE. In addition to the telemetry collected by network monitoring module **502**, which is designed to be quite sparse and lightweight, a state collector agent may send instructions to the various networking elements/devices (e.g., routers, switches) along the path followed by the application traffic (e.g., number of hops, types of links, links state, congestion level, error rates, etc.), to collect more detailed information about their state. Typically, such detailed information could not be collected by network monitoring module **502** in the first place because of scaling issues. More specifically, network monitoring module **502** will typically process all telemetry, whereas a state collector agent (whether a part of network monitoring module **502** or a separate module) may be used selectively obtain information for flows associated with user feedback. In addition, network monitoring module **502** may also interact with other mechanisms, to train the machine learning models of a predictive routing engine, to predict application QoE (e.g., as described in greater detail below) and/or by performing closed-loop control over the network. Should such mechanisms require additional input features, a state collector agent component of network monitoring module **502** could be used to gather this state information.

[0090] As noted above, the user feedback dataset provides feedback on application quality-of-experience (QoE), and combines network and application telemetry with real user feedback, also referred to as "labels", where subjective experience during an application session may be correlated by matching each collected label with the session telemetry recorded prior to the feedback. The labels themselves can be binary (e.g., "thumbs-up-My experience is good."/"thumbs-down-My experience is bad."), and the telemetry can be represented by a sequence of application reports that aggregate the different metrics observed during a fixed-size window (e.g., every minute, one can compute the average and the maximum jitter, loss, latency, bitrate, etc., observed during the last minute) for a given user. Notably, a dataset for machine learning can be built from sequences of fixed length of application reports for each user.

[0091] QoE sequence model **506**, or "QSM", is a machine learning model trained to infer the QoE of a user based on its telemetry represented by the sequence of application metrics. Using machine learning is beneficial for this since the dimensionality of each sequence is very high. For instance, single reports composed of 50 KPIs and sequences of **30** reports correspond to a total of 30*50=1500 input features per sample. This model may be trained in a supervised fashion on the machine learning sequence dataset.

[0092] Because the QSM takes the whole sequence of reports as input, it can learn the time dependencies between the KPIs and the disruption patterns that drive the QoE.

[0093] In a first embodiment herein, the QoE sequence model may be based on gradient-boosted trees (GBT). This architecture often provides state-of-the-art results for classification or regression tasks, especially when the input data is non-homogenous, as is the case within single reports (e.g., loss is a percentage value, latencies are expressed in milliseconds, etc.). An ensemble of several (e.g., 10) GBTs trained with K-fold cross-validation can be used for more robustness. To reduce the dimensionality of the input, an intermediary step can be used: statistics (min, max, average, etc.) can be extracted from the sequences of KPIs instead of passing the raw sequence of reports directly to the model.

[0094] In a second embodiment herein, the QoE sequence model may be an attention deep neural network (DNN). This architecture implements an attention mechanism allowing it to focus its attention on specific reports of the sequence. That is particularly interesting herein since the perceived application experience is conditioned by the moments of poor quality. The techniques herein attempt to make use of the attention to extract the pattern failures that do influence the user QoE. Thus, using an inductive bias to guide the model toward focusing more on these disrupted moments is very efficient.

[0095] This attention DNN can either implement a basic softmax-based attention mechanism (where softmax is the last activation function of a neural network to normalize the output of a network to a probability distribution over predicted output classes), or it can use a more advanced architecture, like the "Transformer" deep learning architecture based on a multi-head attention mechanism, as may be appreciated by those skilled in the art.

[0096] FIG. **6** illustrates an example of a basic attention mechanism **600**, notably with two example reports ("report **1**" and "report **2**") into attention DNNs, though any number may be used (e.g., ten or even more). Each report of the sequence has an input **610** (e.g., KPIs) that are processed by a first shared feed-forward layer **615** that produces a latent vector (values **620**, e.g., "x_1" and "x_2") and then a second shared feed-forward layer **625** that produces a latent score **630** for each report (e.g., "14" and "12"). Then, softmax activation is applied to the scores (latent score **630**) to obtain a softmax attention score (softmax **635**, e.g., "0.88" and "0.12") and each values vector (values **620**) is multiplied by its corresponding softmaxed attention score for an output **640**. As described below, this output **640** from each report may be combined to produce a sum **645** (e.g., "z"), from which a third feed-forward layer **650** produces a prediction **655** (e.g., "y_{pred}"), accordingly.

[0097] Said differently, FIG. **6** illustrates a high-level architecture of an ensemble attention DNN, where each attention DNN contains a sub-model computing a latent representation and an attention weight for each report. The softmax activation is applied to the vector of attention weights to obtain the attention scores. Then, the latent representations of each report are weighted by their corresponding attention scores and summed. This attention mechanism allows the model to discard non-important reports and give more importance to relevant ones. This weighted sum of latent representation is then processed by dense layers to produce the QoE prediction.

[0098] In a refinement of this embodiment, the attention DNN can be pre-trained using unsupervised learning methods (e.g., an auto-encoder) or self-supervised learning methods, to mitigate a cold start problem.

[0099] Returning to FIG. **5**, QoE sequence model **506** is thus used to infer the QoE of a user during an application session. The predicted QoE is then interpreted by the attention interpretability module **508**, or "AIM". Specifically, the AIM interprets each prediction **655** made by the QSM to identify the reports (i.e., the moments in the sequence) that contributed the most to the QoE prediction. The AIM takes as input the QSM and an input sequence of reports and returns a vector containing the attention scores associated with each report of the sequence. The attention score of a report reflects how much the report contributed to the user's QoE.

[0100] In other words, the AIM may compute the sequence of attention scores of the ensemble model by averaging the sequences of attention scores of each DNN of the ensemble. Because of the softmax layer, the resulting vector of attention scores is normalized.

[0101] In one implementation, the AIM can rely on "Shapley values", which are general-purpose scores to explain the predictions of non-linear models, and assess the influence of individual input variables on the output predictions. For a given QoE prediction, the AIM computes the Shapley value of each input feature, where each Shapely value explains how much the feature contributes to the prediction and what is the magnitude of this contribution. A positive Shapley value indicates that the feature contributes positively to the prediction; a negative value means a negative contribution.

[0102] Each feature of the QSM corresponds to a KPI observed over a specific report, i.e., the granularity of the features is the pair (report, KPI). Since the Shapley values are additive, one can compute a Shapley for each report by summing the per-report (or per-KPI) Shapley values. These per-report Shapley values can be interpreted as attention scores, as their absolute value indicates the importance of the report for the prediction. Moreover, the sign of the value indicates if the contribution of the report is positive (i.e., the local quality is better than average) or negative (the local quality is lower than average).

[0103] In a second, more specific implementation, the AIM relies on a QSM that must be an attention DNN (as in the second embodiment of the QSM noted above). In this implementation, the attention scores can be directly extracted from the attention layer of the DNN. For instance, in the case of a basic attention mechanism, the scores correspond to the output of the softmax layer. In this embodiment, the scores are always positive values, the higher the score, the higher the contribution.

[0104] The sequence of attention scores computed by the AIM-combined with the QoE prediction-provides a method to identify the failure mode of a session, as it projects a multi-dimensional sequence of KPIs onto a one-dimensional sequence. As a result, it allows for instance to distinguish between a poor QoE caused by a single local disruption (reflected by a single peak of attention) or by an accumulation of "slightly annoying" moments (in which case the attention is diluted over multiple reports).

[0105] To illustrate the relation between the sequence of attention scores (weights) computed for a given prediction and the underlying telemetry, the techniques herein can help visualize the co-evolution of the attention scores and the

KPIs. FIGS. **7**A-**7**B illustrate an example comparison **700** of attention weights to four key performance indicators (KPIs) over a sequence of thirty application reports (e.g., of one minute each). For instance, the example shown demonstrates a sequence of thirty reports, with attention scores (or weights) denoted as dashed lines (independent of the KPI), and the four KPIs of a video-conferencing application denoted as solid lines. For instance, panel **710** illustrates an example of "audioMainCT" (audio concealment time), panel **720** illustrates an example of "videoMainRxE2ELoss" (video receiver end-to-end loss), panel **730** illustrates "audioMainRTT" (audio round trip time), and panel **740** illustrates "audioMainTxTotalBitRate" (audio transmission total bit rate). As seen in example comparison **700**, the model focused mostly on two specific moments where the user had a very high value for a video conferencing KPI "audioMainCT" () (i.e., panel **710**), resulting in a correct prediction of the QoE, accordingly.

[0106] FIGS. **8**A-**8**B illustrate another example comparison **800** of the co-evolution of attention scores/weights to KPIs, where through examination of panel **810**, panel **820**, panel **830**, and panel **840**, one can observe a spike in attention at x=15, which is correlated with the spike in audio RTT. This indicates that the model focused mostly on the report with the highest RTT.

[0107] FIGS. **9**A-**9**B illustrate still another example comparison **900** with panel **910**, panel **920**, panel **930**, and panel **940**, showing how a user experienced a satisfactory QoE, until five minutes prior to providing feedback, when a significant local disruption occurred. The model allocated 80% of its attention to this moment.

[0108] As a last example, FIGS. **10**A-**10**B illustrate yet another example comparison **1000**, where panel **1010**, panel **1020**, panel **1030**, and panel **1040** demonstrate a situation where the QoE of the user is overall poor (around 200 ms of audio CT, 20% of video loss, 400 ms of audio RTT), with some variance but no huge disruption as in the previous examples. Here it can be seen that the model diluted its attention over all the reports, with attention scores between 2% and 6%.

[0109] The attention score vectors are computed for all the samples of the machine learning sequence dataset and used to fit the attention pattern extractor **510**, or "APE". The APE, in particular, computes clusters of attention score sequences that exhibit similar patterns. Each cluster (pattern) can be assimilated to a specific failure pattern of the application. By analyzing the frequency of each pattern observed over the different paths of the network, one can better understand which are the different failure modes that occur in the network, which is essential to assess and monitor the QoE.

[0110] The APE is based on a sequence clustering algorithm, such as the time-series K-means. It is fitted on the set of attention vectors extracted by the AIM from the machine learning sequence dataset and the QSM. Notably, in a refined embodiment, one can first separate the samples into bins of predicted QoE before applying the clustering model, to extract patterns specific to each level of QoE.

[0111] Each failure pattern can be visualized by plotting a heatmap of the attention score sequences contained in the corresponding cluster. FIG. **11** illustrates an example heatmap **1100** of the Shapely values of sixteen clusters fitted on negative feedbacks. For instance, each identified cluster (e.g., "cluster **0**" through "cluster **15**") may graphically visualize the Shapley values ("shapValue") against time before feedback (e.g., in minutes), with the heatmap indicating a scale of density of values. Each cluster may also indicate a correspondingly predicted QoE ("predQoE") value, such as a percentage value, and a size of the associated data set.

[0112] For closer inspection, FIGS. **12**A-**12**B, in particular, illustrate a visualization of two failure patterns computed by the attention pattern extractor. The visualizations correspond to an attention score ("attentionScore") for each cluster, and an illustrative level of "positivity" (e.g., a percentage). The first visualization **1210** (FIG. **12**A) corresponds to a single peak of attention **1215**, which indicates a local disruption. The second visualization **1220** (FIG. **12**B) is characterized by the attention scores increasing over time (increase **1225**), indicating a QoE that steadily decreases.

[0113] In one embodiment, the previous components are thus applied successively to assess the user's QoE:

[0114] the QSM predicts the QoE corresponding to the user's telemetry;

[0115] the AIM computes the sequence of attention scores associated with the prediction; and

[0116] the APE maps the sequence of attention scores to a specific failure mode (pattern).

[0117] According to one implementation herein, another component, a pattern interpreter module **512**, or "PIM", may be used to expose the insights of the APE to a network administrator. This allows, for example, showing that patterns with more than n disruptions with a specific profile (e.g., a spike of Delay or Loss) lead to a decrease of positivity rate by y % (e.g., where a positivity rate represents the percentage of calls for which the labels were "Good"). This accordingly provides invaluable feedback for network administrators, especially when correlated with network metrics used to refine the network design. To that end, a network management system (NMS), such as an output from network monitoring module **502**, could be used to report the number of link failures (leading to packet loss), QoE congestion events (leading to packet loss, increased jitter, etc.,), sub-optimal routes (leading to high delays) in a given region where the failure patterns have been observed by the APE component. Such correlation leads to adjusting the network design: for example, when observing that the number of disruptions is a key factor driving QoE, the network administrator could decide to adjust the protection/restoration strategy, whereas high jitter patterns could lead to adjusting the QoS policy in that region.

[0118] FIG. **13** illustrates an example simplified procedure for model-based assessment of QoE in accordance with one or more embodiments described herein. For example, a non-generic, specifically configured device (e.g., device **200**, an apparatus) may perform procedure **1300** by executing stored instructions (e.g., process **248**). The procedure **1300** may start at step **1305**, and continues to step **1310**, where, as described in greater detail above, the techniques herein establish a prediction of a quality-of-experience measure from session telemetry regarding execution of an application by one or more users (e.g., where the session telemetry comprises a sequence of application reports that aggregate a plurality of metrics observed during a given period of time for a given user). In particular, as noted above, the prediction is established based on inputting the session telemetry into a machine learning model trained to extract attributes that drive user-based quality-of-experience feedback. As noted above, the machine learning model may

be trained with sequences of past session telemetry and respective user-based feedback to learn time dependencies between certain key performance indicators and disruption patterns to infer quality-of-experience measures from input session telemetries, accordingly.

[0119] As described in greater detail above, in one implementation the techniques herein may use one or more gradient-boosted trees for the machine learning model (e.g., configured with K-fold cross-validation). As mentioned above, the one or more gradient-boosted trees may be configured to use extracted statistics from the session telemetry as inputs to the machine learning model.

[0120] As also described in greater detail above, in another implementation the techniques herein may use an attention deep neural network for the machine learning model. For instance, in this implementation, the techniques herein may implement, by the attention deep neural network, an attention mechanism to cause the machine learning model to focus attention on specific portions of the session telemetry. As noted, the attention mechanism may comprise one of either a softmax-based attention mechanism or a multi-head attention mechanism.

[0121] In step **1315**, the techniques herein may determine one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model. For instance, this may be accomplished in one implementation by determining a sequence of attention scores associated with the prediction and correlated with the one or more attributes (where attention scores reflect how much each of the one or more attributes contributed to the prediction), and selecting particular attributes of the one or more attributes that significantly contributed to the prediction based on having comparatively high respective attention scores. In other implementation, this may be accomplished by computing Shapley values for each of the one or more attributes of the session telemetry that is input into the machine learning model to assess an influence of each of the one or more attributes on the prediction.

[0122] In step **1320**, the techniques herein may map the one or more attributes of the session telemetry that significantly contributed to the prediction to a specific failure pattern from a set of known failure patterns. As explained above, the set of known failure patterns may be based on assimilating clusters of attention score sequences that exhibit similar patterns during training of the machine learning model to respective failure patterns.

[0123] Then, in step **1325**, the techniques herein may mitigate the prediction of the quality-of-experience measure based on the specific failure pattern. In one implementation, mitigating the prediction may comprise correlating the specific failure pattern with one or more network performance indicators, and causing one or more adjustments to a computer network based on correlating the specific failure pattern with the one or more network performance indicators. In another implementation, it may comprise interpreting the one or more attributes and the specific failure pattern to establish one or more insights regarding the quality-of-experience measure, and sharing the one or more insights with an administrator. In still other implementations, mitigating the prediction may be based on providing, via a graphical interface, a representation of the quality-of-experience measure based on one or more of the one or more attributes of the session telemetry, the prediction, and the specific failure pattern. For instance, this representation may be selected from a group consisting of: a visualization of a co-evolution of attention scores and key performance indicators; a heatmap plotting attention scores; a report of network metrics from a given region where the specific failure pattern has been observed; and so on.

[0124] Procedure **1300** may end at step **1330**. Other steps not shown above may be included in the procedure **1300**, such as collecting specific user feedback regarding subjective quality-of-experience for the application, and correlating the specific user feedback against the prediction.

[0125] It should be noted that while certain steps within the procedures above may be optional as described above, the steps shown in the procedures above are merely examples for illustration, and certain other steps may be included or excluded as desired. Further, while a particular order of the steps is shown, this ordering is merely illustrative, and any suitable arrangement of the steps may be utilized without departing from the scope of the embodiments herein. Moreover, while procedures may have been described separately, certain steps from each procedure may be incorporated into each other procedure, and the procedures are not meant to be mutually exclusive.

[0126] In some implementations, an illustrative apparatus herein may comprise: one or more network interfaces to communicate with a network; a processor coupled to the one or more network interfaces and configured to execute one or more processes; and a memory configured to store a process that is executable by the processor, the process comprising: establishing a prediction of a quality-of-experience measure from session telemetry regarding execution of an application by one or more users, the prediction established based on inputting the session telemetry into a machine learning model trained to extract attributes that drive user-based quality-of-experience feedback; determining one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model; mapping the one or more attributes of the session telemetry that significantly contributed to the prediction to a specific failure pattern from a set of known failure patterns; and mitigating the prediction of the quality-of-experience measure based on the specific failure pattern.

[0127] In still other implementations, a tangible, non-transitory, computer-readable medium storing program instructions that cause a device to execute a process comprising: establishing a prediction of a quality-of-experience measure from session telemetry regarding execution of an application by one or more users, the prediction established based on inputting the session telemetry into a machine learning model trained to extract attributes that drive user-based quality-of-experience feedback; determining one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model; mapping the one or more attributes of the session telemetry that significantly contributed to the prediction to a specific failure pattern from a set of known failure patterns; and mitigating the prediction of the quality-of-experience measure based on the specific failure pattern.

[0128] The techniques described herein, therefore, provide for model-based assessment of QoE. In particular, the techniques herein provide real understanding on how failure patterns in the network and/or applications actually drive user experience. That is, to aid in understanding failure patterns governing user experience, QoE models are used (e.g., GBT and attention DNNs) to extract the key attributes

that drive QoE experience, correlate failure patterns with network KPI (events), and adjust the network design to improve the overall user experience.

[0129] Illustratively, the techniques described herein may be performed by hardware, software, and/or firmware, (e.g., an "apparatus") such as in accordance with the QoE assessment process, process **248**, e.g., a "method"), which may include computer-executable instructions executed by the processor(s) **220** to perform functions relating to the techniques described herein, e.g., in conjunction with corresponding processes of other devices in the computer network as described herein (e.g., on agents, controllers, computing devices, servers, etc.). In addition, the components herein may be implemented on a singular device or in a distributed manner, in which case the combination of executing devices can be viewed as their own singular "device" for purposes of executing the process (e.g., process **248**).

[0130] While there have been shown and described illustrative implementations above, it is to be understood that various other adaptations and modifications may be made within the scope of the implementations herein. For example, while certain implementations are described herein with respect to certain types of networks in particular, the techniques are not limited as such and may be used with any computer network, generally, in other implementations. Moreover, while specific technologies, protocols, architectures, schemes, workloads, languages, etc., and associated devices have been shown, other suitable alternatives may be implemented in accordance with the techniques described above. In addition, while certain devices are shown, and with certain functionality being performed on certain devices, other suitable devices and process locations may be used, accordingly. Also, while certain embodiments are described herein with respect to using certain models for particular purposes, the models are not limited as such and may be used for other functions, in other embodiments.

[0131] Moreover, while the present disclosure contains many other specifics, these should not be construed as limitations on the scope of any implementation or of what may be claimed, but rather as descriptions of features that may be specific to particular implementations. Certain features that are described in this document in the context of separate implementations can also be implemented in combination in a single implementation. Conversely, various features that are described in the context of a single implementation can also be implemented in multiple implementations separately or in any suitable sub-combination. Further, although features may be described above as acting in certain combinations and even initially claimed as such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a sub-combination or variation of a sub-combination.

[0132] Similarly, while operations are depicted in the drawings in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. Moreover, the separation of various system components in the implementations described in the present disclosure should not be understood as requiring such separation in all implementations.

[0133] The foregoing description has been directed to specific implementations. It will be apparent, however, that other variations and modifications may be made to the described implementations, with the attainment of some or all of their advantages. For instance, it is expressly contemplated that the components and/or elements described herein can be implemented as software being stored on a tangible (non-transitory) computer-readable medium (e.g., disks/CDs/RAM/EEPROM/etc.) having program instructions executing on a computer, hardware, firmware, or a combination thereof. Accordingly, this description is to be taken only by way of example and not to otherwise limit the scope of the implementations herein. Therefore, it is the object of the appended claims to cover all such variations and modifications as come within the true intent and scope of the implementations herein.

What is claimed is:

1. A method, comprising:

establishing, by a device, a prediction of a quality-of-experience measure from session telemetry regarding execution of an application by one or more users, the prediction established based on inputting the session telemetry into a machine learning model trained to extract attributes that drive user-based quality-of-experience feedback;

determining, by the device, one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model;

mapping, by the device, the one or more attributes of the session telemetry that significantly contributed to the prediction to a specific failure pattern from a set of known failure patterns; and

mitigating, by the device, the prediction of the quality-of-experience measure based on the specific failure pattern.

2. The method of claim **1**, further comprising:

collecting specific user feedback regarding subjective quality-of-experience for the application; and

correlating the specific user feedback against the prediction.

3. The method of claim **1**, wherein the session telemetry comprises a sequence of application reports that aggregate a plurality of metrics observed during a given period of time for a given user.

4. The method of claim **1**, wherein the machine learning model is trained with sequences of past session telemetry and respective user-based feedback to learn time dependencies between certain key performance indicators and disruption patterns to infer quality-of-experience measures from input session telemetries.

5. The method of claim **1**, further comprising:

using one or more gradient-boosted trees for the machine learning model.

6. The method of claim **5**, wherein the one or more gradient-boosted trees are configured with K-fold cross-validation.

7. The method of claim **5**, wherein the one or more gradient-boosted trees are configured to use extracted statistics from the session telemetry as inputs to the machine learning model.

8. The method of claim **1**, further comprising:

using an attention deep neural network for the machine learning model.

9. The method of claim 8, further comprising:

implementing, by the attention deep neural network, an attention mechanism to cause the machine learning model to focus attention on specific portions of the session telemetry.

10. The method of claim 9, wherein the attention mechanism comprises one of either a softmax-based attention mechanism or a multi-head attention mechanism.

11. The method of claim 1, wherein determining the one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model comprises:

determining a sequence of attention scores associated with the prediction and correlated with the one or more attributes, wherein attention scores reflect how much each of the one or more attributes contributed to the prediction; and

selecting particular attributes of the one or more attributes that significantly contributed to the prediction based on having comparatively high respective attention scores.

12. The method of claim 1, wherein determining the one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model comprises:

computing Shapley values for each of the one or more attributes of the session telemetry that is input into the machine learning model to assess an influence of each of the one or more attributes on the prediction.

13. The method of claim 1, wherein the set of known failure patterns is based on assimilating clusters of attention score sequences that exhibit similar patterns during training of the machine learning model to respective failure patterns.

14. The method of claim 1, wherein mitigating comprises:

correlating the specific failure pattern with one or more network performance indicators; and

causing one or more adjustments to a computer network based on correlating the specific failure pattern with the one or more network performance indicators.

15. The method of claim 1, wherein mitigating comprises:

interpreting the one or more attributes and the specific failure pattern to establish one or more insights regarding the quality-of-experience measure; and

sharing the one or more insights with an administrator.

16. The method of claim 1, wherein mitigating comprises:

providing, via a graphical interface, a representation of the quality-of-experience measure based on one or more of the one or more attributes of the session telemetry, the prediction, and the specific failure pattern, the representation selected from a group consisting of: a visualization of a co-evolution of attention scores and key performance indicators; a heatmap plotting attention scores; and a report of network metrics from a given region where the specific failure pattern has been observed.

17. An apparatus, comprising:

one or more network interfaces to communicate with a network;

a processor coupled to the one or more network interfaces and configured to execute one or more processes; and

a memory configured to store a process that is executable by the processor, the process comprising:

establishing a prediction of a quality-of-experience measure from session telemetry regarding execution of an application by one or more users, the prediction established based on inputting the session telemetry into a machine learning model trained to extract attributes that drive user-based quality-of-experience feedback;

determining one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model;

mapping the one or more attributes of the session telemetry that significantly contributed to the prediction to a specific failure pattern from a set of known failure patterns; and

mitigating the prediction of the quality-of-experience measure based on the specific failure pattern.

18. The apparatus of claim 17, wherein the machine learning model comprises one of either one or more gradient-boosted trees or an attention deep neural network.

19. The apparatus of claim 17, wherein mitigating comprises:

correlating the specific failure pattern with one or more network performance indicators; and

causing one or more adjustments to a computer network based on correlating the specific failure pattern with the one or more network performance indicators.

20. A tangible, non-transitory, computer-readable medium storing program instructions that cause a device to execute a process comprising:

establishing a prediction of a quality-of-experience measure from session telemetry regarding execution of an application by one or more users, the prediction established based on inputting the session telemetry into a machine learning model trained to extract attributes that drive user-based quality-of-experience feedback;

determining one or more attributes of the session telemetry that significantly contributed to the prediction from the machine learning model;

mapping the one or more attributes of the session telemetry that significantly contributed to the prediction to a specific failure pattern from a set of known failure patterns; and

mitigating the prediction of the quality-of-experience measure based on the specific failure pattern.

* * * * *